

COMPILATION

Design and implementation of a compiler for the E language

CYBER – Lab handout

1 Purpose of these lab sessions

During these lab sessions, you will create a compiler for a small programming language, the E language. The E language is a derivative of the C language. We will first focus on compiling a basic version of the E language, which we will later enhance during project slots. Your compiler will consist of a lexical analyzer (*lexer*), a syntax analyzer (*parser*), and then several compilation passes that will transform E programs into progressively lower-level languages, up to the generation of RISC-V assembly code, which will finally be assembled by existing assemblers and can be executed on your machines (using *qemu*).

As we are going to see, the E language is relatively small, allowing you to complete it within the lab time allotted to you. However, it allows us to illustrate a large number of fundamental compilation concepts. In case you find that the language is too small, or that the suggested compilation and optimization passes are not sufficient, we will provide you with a list of possible improvements that you can implement.

[Section 2](#) presents the architecture of the compiler that you are going to design, including the data structures to use and the different intermediate languages. [Section 3](#) presents the testing infrastructure that will assist you in debugging your compiler. The following sections describe the work that you will have to do during the lab sessions. The division into lab sessions is given as a guideline. If you have not finished the required work at the end of a lab session, you will be able to use a part of the following session to finish it. Try not to fall *too far* behind. For the lexer and parser, there is a shortcut you can use if you're stuck, don't hesitate to ask us.

You will find the skeleton associated with this lab at the following address:

<https://gitlab-research.centralesupelec.fr/cidre-public/compilation/infosec-ecomp>

You will want to work on this project in a git repository to share your code with your partner. To do so, create a blank repository on the platform of your choice (a CentraleSupélec gitlab, a github, something else), then use the following procedure:

```
$ git clone git@gitlab-research.centralesupelec.fr:cidre-public/compilation/
infosec-ecomp.git
$ cd infosec-ecomp
$ git remote rename origin upstream
$ git remote add origin git@example.com:your-account/your-repository.git
```

You will then be able to use this git repository normally (commit / push / pull) as you are used to. So that we can help you, please give us read access to your git repository. (Frédéric Tronel, Pierre Wilke and Samuel Hiron)

We may update the upstream repo during the lab sessions to update this handout, or fix potential errors. When that happens, we will notify you and you will need to commit your changes to your own git repository before doing:

```
$ git pull upstream master
```

This will retrieve the changes that we will have pushed. (Most of the time, it will not be to bother you but to provide you with more robust and better-documented code.)

2 Compiler organization

Figure 1 provides an overview of the compiler structure that you will create. From a source file *.e*, the lexical analyzer (or *lexer*) will generate a stream of lexemes (or *tokens*). This stream will be given to the syntax analyzer (or *parser*), which will have to generate an *Abstract Syntax Tree* (AST). The AST will be transformed into a sequence of intermediate languages:

- an E program, which is simply a formal representation, in OCaml, of the source program;
- a CFG program (*Control-Flow Graph*) ;
- an RTL program (*Register Transfer Language*) ;
- a Linear program;
- an LTL program (*Location Transfer Language*) ;
- a RISC-V Assembly program.

Each of these intermediate languages is detailed in the lab topic to which it relates, and is illustrated by the following example.

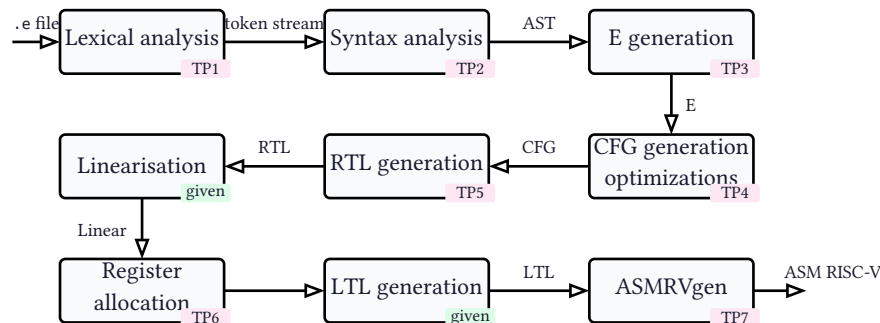


Figure 1: Overview of the compiler structure

```

main(a,b){
  while(b != 0){
    t = b;
    b = a % b;
    a = t;
  }
  return a;
}

```

Figure 2: E program

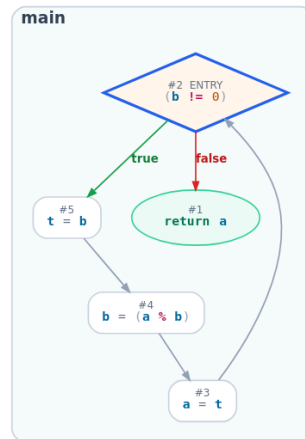


Figure 3: Corresponding CFG

```

main(r0, r1):
  main_2:
    r2 <- 0
    r1 != r2 ? jmp main_5
    jmp main_1

  main_3:
    r0 <- r3
    jmp main_2

  main_5:
    r3 <- r1
    jmp main_4

  main_4:
    r4 <- %(r0, r1)
    r1 <- r4
    jmp main_3

  main_1:
    ret r0

```

Figure 4: RTL program

```

main:
  # Prologue
  addi sp, sp, -8
  sd ra, 0(sp)
  addi sp, sp, -8
  sd s0, 0(sp)
  mv s0, sp
  addi sp, sp, -24
  # Linear source: main_2:
main_2:
  # Linear source: r2 <- 0
  li t0, 0
  sd t0, -8(s0)
  # Linear source: r1 != r2 ? jmp main_5
  ld t1, -8(s0)
  bne a1, t1, main_5
  # Linear source: jmp main_1
  j main_1
  # Linear source: main_5:
main_5:
  # Linear source: r3 <- r1
  mv t0, a1
  sd t0, -16(s0)
  # Linear source: r4 <- %(r0, r1)
  remu t0, a0, a1
  sd t0, -24(s0)

  # Linear source: r1 <- r4
  ld t0, -24(s0)
  mv a1, t0
  # Linear source: r0 <- r3
  ld t0, -16(s0)
  mv a0, t0
  # Linear source: jmp main_2
  j main_2
  # Linear source: main_1:
main_1:
  # Linear source: ret r0
  mv a0, a0
  j main_6
  # Epilogue
main_6:
  mv sp, s0
  ld s0, 0(sp)
  addi sp, sp, 8
  ld ra, 0(sp)
  addi sp, sp, 8
  jr ra

```

Figure 5: RISC-V assembly program

3 Tests

Expected test coverage

The default `make test` command runs `ecomp-tests/basic/`. As the language is extended, keep earlier suites passing and add the suite for each new feature: `funcall`, `type_basic` and `char`, `ptr`, `structs`, `array`, `globals`, `invader_features`, and finally the `invader` integration program.

We provide a separate repository that contains a set of tools allowing you to test your compiler. The folders `basic`, `funcall`, `type_basic`, `type_funcall`, `char`, `ptr`, `structs`, `array`, `globals` and `invader` contain E programs allowing you to test the features corresponding to their directory names. All the way through to [Section 10](#), we will focus exclusively on the tests in the `basic` directory.

For simplicity, please clone the following repository next to the `infosec-ecomp` repository you cloned earlier.

```
$ git clone git@gitlab-research.centralesupelec.fr:cidre-public/compilation/
ecomp-tests.git
```

You should now have the directories `infosec-ecomp` and `ecomp-tests` in the same directory.

The test harness uses the `.ecomp-section` file at the root of your compiler repository to distinguish work still in progress from regressions in a lab you have completed. This file records the **last complete lab**, not the lab you are currently working on. At the beginning, it must contain the single line `none`. After finishing an entire lab, replace that line with the corresponding value:

Value	Last completed lab
<code>lexing</code>	TP1: lexical analysis
<code>parsing</code>	TP2: parsing
<code>elang</code>	TP3: E generation and interpreter
<code>cfg</code>	TP4: CFG liveness and dead-code elimination
<code>rtl</code>	TP5: RTL generation
<code>regalloc</code>	TP6: register allocation
<code>funcalls</code>	Function calls
<code>typing</code>	Typing
<code>pointers</code>	Pointers
<code>structures</code>	Structures
<code>arrays</code>	Arrays
<code>globals</code>	Globals
<code>invaders</code>	Space Invaders (final checkpoint)

For example, while you are working through TP4, keep `elang` in the file. Only after completing TP4 should you run:

```
$ printf 'cfg\n' > .ecomp-section
```

Commit this one-line update with the completed lab. A missing, empty, or invalid file does not prevent the tests from running, but the report will warn you and cannot distinguish expected work-in-progress failures from regressions.

During later lab sessions, you will improve your compiler to extend its language towards C. For these extensions, you will be able to use the tests provided in the various other directories.

For each file `file.e`, we provide a file `file.e.test.json` that records the expected behaviour of that file, i.e. whether it should compile cleanly or not, and what result is expected for two set of parameters. You can test that your compiler produces the expected results by running `make test` from the root of your `infosec-ecomp` project.

The test results will be collected in `.test-results/results.html`, which you can view with your preferred web browser. The results are presented in the form of tables where the rows correspond to the programs tested and the columns to the results obtained at different stages of compilation. The program names are clickable to get more information on the progress of their compilation and their execution in each of the different intermediate languages.

To compile a program individually, use `ecomp`:

```
# In infosec-ecomp
$ ./ecomp -f ../ecomp-tests/basic/1_or_3a.e
$ ./ecomp --help
```

Running an intermediate artifact is the responsibility of `ecomp-run`; in most cases the per-file report produced by `make test` is the simplest way to inspect both compilation and execution.

4 TP1: Lexical analyzer

Expected test coverage

The lexer should pass the lexer checks for `ecomp-tests/basic/`. The lexer files in later suites are additional checks when their syntax is introduced.

The goal of this lab session is to create a lexical analyzer for the E language. This session is an opportunity to implement the algorithm seen in class for creating a lexical analyzer. We remind you that it is based on the use of a deterministic finite-state automaton. In order to speed up your development, we will provide you with part of the code and suggest an organization for your code.

4.1 Useful functions of the OCaml standard library

The complete documentation is available online at: <https://ocaml.org/manual/5.3/api/index.html>

The framework uses the OCaml standard library. A few small supplementary definitions, including the concrete set modules used below, are provided in `src/collections.ml`.

Functions on lists: <https://ocaml.org/manual/5.3/api/List.html>

- `List.mem`: checks if an element belongs to a list

- `List.fold_left`: applies a function `f` to each element of a list `l` while storing the result in `acc`
- `List.map`: applies a function to each element of a list
- `List.filter_map`: filters the elements of a list and applies a function to the elements that are kept.
- `List.mem_assoc`: given an association list (a list whose elements are pairs (k, v) where k corresponds to a key and v to a value), determines if a key is present in the list.
- `List.assoc_opt`: given an association list, returns the value associated with a key (or `None` if this key is not defined)
- `List.remove_assoc`: removes the associations for a given key in an association list
- `List.iter`: iterates a function of type `unit` over each element of a list
- `List.rev`: reverses a list (the first elements become the last)

Functions on sets `Set`: <https://ocaml.org/manual/5.3/api/Set.S.html>

The standard-library `Set` module is functorized: a concrete module must specify the type and ordering of its elements. The framework defines `CharSet`, `IntSet`, and `StringSet` in `Collections`. In the lexer exercises, `Set` is a local alias for `Collections.IntSet`, while character sets use `CharSet` explicitly.

- `Set.union`: union of two sets
- `Set.add`: adding an element to a set
- `Set.mem`: checks if an element belongs to a set
- `Set.empty`: the empty set
- `Set.is_empty`: checks if a set is empty
- `Set.singleton`: constructs a set containing a single element
- `Set.fold`: like `List.fold_right`, but here the order of elements cannot be taken into account (no order in a set)
- `Set.exists`: checks if there exists an element of a set that satisfies a predicate.

Functions on hash tables `Hashtbl`: <https://ocaml.org/manual/5.3/api/Hashtbl.html>

- `Hashtbl.find_opt`: searches for the value associated with a key in a hash table.
- `Hashtbl.replace`: modifies or creates a key/value association in a hash table.
- `Hashtbl.create`: creates an empty hash table
- `Hashtbl.to_seq_keys`: returns a sequence of the keys for which an association is known in a hash table. Use `List.of_seq` when a list is needed.

4.2 Tests

To test the code of the different functions of the lexical analyzer, independently of the rest of the compiler, you can run the following command from the root of your project:

```
$ make -C src test_lexer
make: Entering directory '<PATH>/src'
cd .. && dune exec --root . ./src/test_lexer.exe
0: ([w]).([h]).([i]).([l]).([e]).(Eps)))
1: ([i]).([f]).(Eps))
2: ([ABCDEFGHJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxy]).
([0123456789ABCDEFGHIJKLMNPOQRSTUVWXYZ_abcdefghijklmnopqrstuvwxy]*)
===== NFA
States :
```

Initial states :

Final states :

```
[K0] epsilon_closure 1 : got , expected 1_2_3
[K0] epsilon_closure 2 : got , expected 2
[K0] epsilon_closure 3 : got , expected 2_3
[K0] epsilon_closure 4 : got , expected 4
[K0] dfa_initial_state : got , expected 1_2_3
[K0] min_priority 1 : got None, expected Some (SYM_WHILE)
[K0] min_priority 2 : got None, expected Some (SYM_IDENTIFIER(bla))
[K0] min_priority 3 : got None, expected Some (SYM_WHILE)
[OK] min_priority 4
[K0] dfa states : got {}, expected {{1,2,3}, {2,4}, {2}}
*** NFA n1 : 'hello'
[K0] nfa_accepts hello = false
[OK] nfa_accepts bonjour = false
*** NFA n2 : 'bonjour'
[K0] nfa_accepts hello = false
[K0] nfa_accepts bonjour = false
*** NFA n3 : n1 | n2
[K0] nfa_accepts hello = false
[K0] nfa_accepts bonjour = false
[OK] nfa_accepts buongiorno = false
*** NFA n4 : n1 . n2
[K0] nfa_accepts hello = false
[K0] nfa_accepts bonjour = false
[K0] nfa_accepts hellobonjour = false
[OK] nfa_accepts bonjourhello = false
*** NFA n5 : n1*
[K0] nfa_accepts = false
[K0] nfa_accepts hello = false
[K0] nfa_accepts hellohello = false
[OK] nfa_accepts hellobonjour = false
*** NFA n6 : n3*
[K0] nfa_accepts = false
[K0] nfa_accepts hello = false
[K0] nfa_accepts hellohello = false
[K0] nfa_accepts hellobonjour = false
[K0] nfa_accepts hellobonjourhello = false
[K0] nfa_accepts bonjourbonjourbonjourhello = false
[OK] nfa_accepts bonjlo = false
dot -Tsvg /tmp/dfa.dot -o /tmp/dfa.svg
dot -Tsvg /tmp/nfa.dot -o /tmp/nfa.svg
make: Leaving directory '<PATH>/src'
```

This has the effect of compiling the file `src/test_lexer.ml` and running it. This file calls the functions that you will define gradually during the lab in `src/lexer_generator.ml` and compares their result with the expected result.

This test generates graphs in dot format, then converts them to SVG to display them in a human-readable form. If this is not already the case, install the package `graphviz` which contains the utility `dot`. The compiler will also need it later, to build per-file reports. You can

examine the generated nondeterministic and deterministic automata by opening the files `/tmp/nfa.svg` and `/tmp/dfa.svg`, for example with your web browser.

To compile and test your code within the compiler, simply run `make test` in the root directory of your project. The test results are stored in `.test-results/results.html`.

Example expected results:

```
$ cat ../ecompile-tests/basic/just_a_variable_37.e
main(){
    just_a_variable = 37;
    return just_a_variable;
}
```

At the beginning of the lab, in `.test-results/results.html`:

```
Lexing error:
Lexer failed to recognize string starting with 'main(){
    just_a_var'
```

At the end of the lab:

```
SYM_IDENTIFIER(main)
SYM_LPARENTHESIS
SYM_RPARENTHESIS
SYM_LBRACE
SYM_IDENTIFIER(just_a_variable)
SYM_ASSIGN
SYM_INTEGER(37)
SYM_SEMICOLON
SYM_RETURN
SYM_IDENTIFIER(just_a_variable)
SYM_SEMICOLON
SYM_RBRACE
SYM_EOF
```

When you call `make test`, your compiler is run on the `ecompile-tests/basic/*.e` files. For the moment, `.test-results/results.html` indicates that all these tests fail at lexical analysis since your parser has not yet been written. As you progress through the lab sessions, this file will provide you with more and more information, including the results of lexical and syntactic analysis as well as the execution results of each test program at different stages in the compilation chain. This will be a good way to validate the correctness of your compilation passes.

You can also run the compiler *manually*, that is, without going through the `make test` command:

```
# build the compiler
$ make
# launch the compiler
$ ./ecompile -f ../ecompile-tests/basic/just_a_variable_37.e -tokens-save -
```

to run the compiler on the file `../ecompile-tests/basic/just_a_variable_37.e` and display the recognized tokens. (The `-` at the end of the command line indicates that we want to display

the tokens on the standard output. If we want to write them to a file, we will replace this - with the name of the file.)

4.3 Work to be done

The development of our lexical analyzer takes place in three steps.

1. First, specify regular expressions that recognize the tokens of the E language.
2. Then, an NFA (*Non-deterministic Finite Automaton*) can be generated for these regular expressions.
3. Finally, this NFA will be transformed into a DFA (*Deterministic Finite Automaton*), which the lexer will use to recognize input tokens and associate them with their token kinds.

The work that you will carry out during this session will take place in the files `src/lexer_generator.ml` and `src/e_regexp.ml`.

4.3.1 Regular expressions of the E language

A first task is to give the lexer different regular expressions allowing it to identify the keywords and variable names of the E language. To familiarize yourself with the E language, do not hesitate to browse the `ecomp-tests/basic` directory, where about thirty examples are provided for you.

Open the file `src/e_regexp.ml`. It defines the type of regular expressions `regexp`. The different variants correspond to the different constructions of regular expressions seen in class. (Except for the variant `Charset s`, which directly represents a character among a set, rather than using many `Alt ( , , )`, for reasons of practicality and performance.)

Next, an alphabet is defined, consisting of all lowercase and uppercase letters, numbers, as well as a certain number of non-alphanumeric symbols.

Next comes the definition of `list_regexp`, the list of regular expressions, each corresponding to a lexeme of our language E. This list is of type `(regexp * (string -> token option)) list`. Each element of the list is therefore a pair (r, t) where r is a regular expression (type `regexp`) and t is a function that constructs, from the string captured by the regular expression, a lexeme (type `token`). For lexemes of keyword type, which are constant, the string passed as a parameter to the function t will have no effect. On the other hand, for lexemes like `SYM_IDENTIFIER`, the string will correspond to the identifier itself.

A certain number of regular expressions associated with lexemes have already been built for you. You still need to complete this list for a number of keywords (where the regular expression `Eps` is used, while you complete the code).

Question 4.1

Complete the function `list_regexp` of the file `src/e_regexp.ml` by replacing the regex `Eps` with an appropriate regular expression. Note that the list of lexemes (type `token`) is available in the file `src/symbols.ml`.

4.3.2 Regular expressions in NFAs

We now want to produce the NFA corresponding to the regular expressions used to analyze the language.

In the file `src/lexer_generator.ml`, the type `nfa` describes what a non-deterministic finite automaton (NFA) is. The field `nfa_final` is a list of pairs (q, t) where q is a state of the automaton, which is declared as final, and t is a function of type `string -> token option` (the same type as the one seen in the file `src/e_regex.ml`). This function constructs, when possible, a lexeme from the string recognized by the automaton.

Question 4.2

Write the functions `cat_nfa`, `alt_nfa` and `star_nfa` from file `src/lexer_generator.ml`. These functions allow, respectively, the concatenation, union, and repetition of automata `nfa`. *The type `nfa` is described and commented at the beginning of file.* These functions must reproduce the method of translating regular expressions into non-deterministic finite automata presented in class. The function `star_nfa` is parameterized by a function t of type `string -> token option`, which must be associated with the final state of the constructed automaton.

Question 4.3

Complete the function `nfa_of_regex` which produces an NFA from a regular expression. The different parameters of this function are described in the skeleton of the lab.

The cases `Eps` and `Charset c` are given as examples. Handle the remaining variants of `regex`.

You can test your implementation here by running, from the `src` directory:

```
$ make test_lexer
```

This will print a number of information, most of which are not relevant so far. The file `"/tmp/nfa.dot"` should now have been generated, together with a file `"/tmp/nfa.svg"`. You can open the latter and make sure what you visualize corresponds to what you expected, for the list of regular expressions you defined at the beginning of the file `src/test_lexer.ml`, that is, an NFA that recognizes the `if` and `while` keywords, as well as identifiers.

4.3.3 Determinization of an NFA into a DFA

In this section, we are interested in transforming our NFA into a DFA, which will allow us to build our lexical analyzer, following the steps described in class. The type `dfa` used in this section is defined and commented on in the file `src/lexer_generator.ml`.

Question 4.4

Complete the function `epsilon_closure`, which returns the set of states reachable by ϵ -transitions from a state of an automaton `nfa`.

In particular, you will need to write the internal function `traversal` (`visited: nfa_state set`) (`s: nfa_state`) : `nfa_state set`, following only the ϵ transitions. The parameter `visited` of this function contains the set of states already visited and `s` is the state from which we want to start the traversal.

The few tests related to the function `epsilon_closure` should display OK, after a `make test_lexer`.

Question 4.5

Complete the function `epsilon_closure_set` which returns the set of states reachable by ε -transitions from a set of states of an automaton `nfa`.

Question 4.6

Write the function `dfa_initial_state` which constructs the initial state of the DFA corresponding to the NFA `n`.

We now have the initial state of our DFA. We will now build its transition table. This table is constructed recursively using the function `build_dfa_table`. This function takes as a parameter a partially constructed table `table`. This table is a hash table where the keys are `dfa_state` and the values are lists of pairs (c, s) where c is a character from the alphabet and s is the state to which one transitions when reading this character. The other parameters of the function are `n`, the NFA that we want to determinize, and `ds`, the state of the DFA currently being built from which we want to add transitions.

Part of the code for this function is provided to you. If the state `in` has already been processed, we return `()`, *i.e.* we do nothing; we are done! Otherwise, we must calculate the set of possible transitions from this state `in`. The procedure for constructing these transitions was covered in class and is detailed in a comment above this function. A number of helper functions, which will be useful to you, have been defined for you.

Question 4.7

Build the transitions expression of the `build_dfa_table` function. This function allows building the transition table of a `dfa` from a `nfa`. The different steps for building this transition table are specified in the comments located above the function.

We now have the initial state of our DFA, as well as its transition table. All that remains is to determine the final states, and our DFA will be complete!

Question 4.8

Complete the functions `min_priority` and `dfa_final_states` allowing the definition of the final states of our `dfa`. The final states of a `dfa` correspond to the states that contain at least one final state of a `nfa`. The conversion function associated with a final state is obtained by using `min_priority`. The purpose of `min_priority` is explained in a comment in the skeleton.

Question 4.9

Complete the transition function `make_dfa_step` using the transition table previously built with `build_dfa_table`. This function should be very simple, based on the table built previously.

The function `dfa_of_nfa`, which constructs a DFA from an NFA, is provided to you. It is simply a matter of using all the functions you have written so far.

You can now restart `make test_lexer` and observe the file `/tmp/dfa.svg` that will have been generated. It should look like the DFA obtained at the very end of the course.

4.3.4 Obtaining lexemes using the DFA

We have now obtained a DFA capable of recognizing the keywords and variables of the E language. We now want our DFA to break down the character strings of an E program into a series of lexemes/tokens defined in the file `src/symbols.ml`.

Question 4.10

Complete the `tokenize_one` function. It contains a recursive `recognize` function which performs transitions in the DFA as long as possible and returns a token when it succeeds.

As before, many details are given to you in the comments of the file `src/lexer_generator.ml`.

Note: you will need the function `string_of_char_list` which transforms a `char list` into `string`

The last steps, namely

- the function `tokenize_all` which repeats `tokenize_one` as long as there are lexemes to read;
- the function `tokenize_file` which transforms a file name into the list of lexemes that are recognized in this file; and
- the pieces of code in `main.ml` that call the lexer; are already written for you!

If all goes well, a `make test` now shows you plenty of errors, but only syntax 😊

5 TP2: Parser**Expected test coverage**

The parser should accept all valid programs in `ecomp-tests/basic/` and reject the malformed programs whose expected errors are recorded there.

During the previous lab, you wrote a lexical analyzer for the E language, and thus obtained, from a `.e` source file, a stream of lexemes. The goal of this lab is to build a syntax analyzer. To do this, you will need to write the grammar of the E language in a specific format and use a syntax analyzer generator.

5.1 ALPAGA: An LL(1) Parser Generator

There are a number of parser generators, the best known being yacc, (*yet another compiler compiler*), and bison (which generate C code), their cousin ocaml yacc (which generates OCaml code), menhir (which also generates OCaml code, but also Rocq!), ANTLR (ANother Tool for Language Recognition, written in Java and which generates Java, C#, Python, JavaScript, Go, C++, and Swift). All these tools accept a grammar as input, in a particular format, and produce source code that traverses the stream of lexemes and produces an abstract syntax tree.

In order to have fine control over the generated parser, and to be able to export a certain number of useful pieces of information when writing the grammar, we chose to build our own tool, and thus add ALPAGA (*An LL(1) PARser GenerAtor*) to the bestiary of parser generators. ALPAGA is written in OCaml and produces OCaml code which can be integrated into your compiler.

In addition to the parser code, ALPAGA produces an HTML file that contains a number of interesting pieces of information. Let us look, for example, at the documentation generated by a very small grammar.

Grammar

(1) S	→ <u>EXPR</u> SYM_EOF
(2) EXPR	→ <u>IDENTIFIER</u>
(3)	→ <u>INTEGER</u>
(4)	→ <u>EXPR</u> SYM_PLUS <u>EXPR</u>
(5)	→ <u>EXPR</u> SYM_ASTERISK <u>EXPR</u>
(6) IDENTIFIER	→ SYM_IDENTIFIER
(7) INTEGER	→ SYM_INTEGER

Figure 6: Grammar (clickable)

First Table

Non-terminal	First
S	SYM_IDENTIFIER SYM_INTEGER
EXPR	SYM_IDENTIFIER SYM_INTEGER
IDENTIFIER	SYM_IDENTIFIER
INTEGER	SYM_INTEGER

Figure 7: FIRST relation

Prediction Table

	SYM_EOF	SYM_PLUS	SYM_ASTERISK	SYM_INTEGER	SYM_IDENTIFIER
S				1	1
EXPR				3 4 5	2 4 5
IDENTIFIER					6
INTEGER				7	

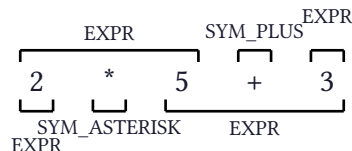
Figure 8: LL table

[Figure 6](#) shows the grammar that we have written, where each rule is numbered, and each non-terminal of the grammar is a link to the set of rules associated with that non-terminal. This seems trivial for this very small grammar, but your grammar will not fit on a single screen and this feature will then be greatly appreciated.

[Figure 7](#) gives, for each non-terminal, the set of terminals that can begin this non-terminal. We see that the non-terminals S and EXPR accept an identifier or an integer as the first lexeme. Similar tables exist for the Null and Follow functions, discussed in class.

Finally, [Figure 8](#) shows the prediction table that will serve as a matrix for the generated parser. Each row corresponds to a non-terminal that we want to parse. S is the conventional name for the starting rule of a grammar, which is called the *start symbol* (or axiom) of the grammar. Each column corresponds to a terminal (lexeme, token, symbol) coming from the lexer. If the (NT, T) cell is empty, it means that the non-terminal NT cannot start with the terminal T , in other words $T \not\in \text{First}(NT)$. If the cell contains the number n , it means that the rule with that number must be applied. If the cell contains several numbers, there is a conflict; our grammar is ambiguous.

Indeed, how should one analyze the expression $2 * 5 + 3$?



Both derivations are grammatically correct, but they give two different results: $(2 * 5) + 3$ on one side, which equals 13, and $2 * (5 + 3)$ on the other, which equals 16. Of course, you learned at school that addition takes precedence over multiplication, and that is why the first solution is the correct one. To explain this to our grammar, as seen in class, it is necessary to write the grammar differently, as in the following grammar. In the accompanying LL table, the rule numbers in blue correspond to the rules that can be applied because the given terminal can start this non-terminal ($t \in \text{First}(nt)$); the numbers in red correspond to the rules that can be applied when the non-terminal is *nullable* and the terminal can *follow* this non-terminal ($\text{Null}(nt) \wedge t \in \text{Follow}(nt)$).

Grammar

(1) S	→ <u>EXPR</u> <u>SYM_EOF</u>
(2) EXPR	→ <u>TERM</u> <u>TERMS</u>
(3) TERM	→ <u>FACTOR</u> <u>FACTORS</u>
(4) FACTOR	→ <u>IDENTIFIER</u>
(5)	→ <u>INTEGER</u>
(6) IDENTIFIER	→ <u>SYM_IDENTIFIER</u>
(7) INTEGER	→ <u>SYM_INTEGER</u>
(8) FACTORS	→ <u>SYM_ASTERISK</u> <u>FACTOR</u> <u>FACTORS</u>
(9)	→ ϵ
(10) TERMS	→ <u>SYM_PLUS</u> <u>TERM</u> <u>TERMS</u>
(11)	→ ϵ

Figure 9: Grammar (clickable)

First Table

Non-terminal	First
S	SYM_IDENTIFIER SYM_INTEGER
EXPR	SYM_IDENTIFIER SYM_INTEGER
TERM	SYM_IDENTIFIER SYM_INTEGER
FACTOR	SYM_IDENTIFIER SYM_INTEGER
IDENTIFIER	SYM_IDENTIFIER
INTEGER	SYM_INTEGER
FACTORS	SYM_ASTERISK
TERMS	SYM_PLUS

Figure 10: FIRST relation

As can be seen, the grammar is more complicated to write, less natural, but unambiguous, and the generated table is conflict-free.

5.1.1 Grammar format

Here is the grammar file given to ALPAGA for the previous example.

```
tokens SYM_EOF SYM_IDENTIFIER<string> SYM_INTEGER<int> SYM_PLUS SYM_ASTERISK
non-terminals S EXPR TERM TERMS FACTOR FACTORS INTEGER IDENTIFIER
axiom S
rules
S -> EXPR SYM_EOF
IDENTIFIER -> SYM_IDENTIFIER
INTEGER -> SYM_INTEGER
EXPR -> TERM TERMS
TERM -> FACTOR FACTORS
TERMS -> SYM_PLUS TERM TERMS
TERMS ->
FACTOR -> IDENTIFIER
FACTOR -> INTEGER
```

Prediction Table

	SYM_EOF	SYM_PLUS	SYM_ASTERISK	SYM_INTEGER	SYM_IDENTIFIER
S				<u>1</u>	<u>1</u>
EXPR				<u>2</u>	<u>2</u>
TERM				<u>3</u>	<u>3</u>
FACTOR				<u>5</u>	<u>4</u>
IDENTIFIER					<u>6</u>
INTEGER				<u>7</u>	
FACTORS	<u>9</u>	<u>9</u>	<u>8</u>		
TERMS	<u>11</u>	<u>10</u>			

Figure 11: LL table

```
FACTORS -> SYM_ASTERISK FACTOR FACTORS
FACTORS ->
```

The file format is therefore as follows:

- Terminal declarations (tokens). The terminals that will be used are declared. In our case, these will be all the symbols (elements of type token from the file `src/symbols.ml`) generated in the previous lab. The keyword tokens is placed followed by the list of tokens. Several such lines can be given.
Note that for non-constant lexemes (`SYM_IDENTIFIER` and `SYM_INTEGER`) the type of the parameter is specified in angle brackets: `<string>` or `<int>`.
- Declarations of non-terminals. Similarly, we declare the list of non-terminals that we are going to recognize, one after the other, after the keyword `non-terminals`.
- Declaration of the grammar axiom: `axiom S`
- The keyword rules. This delimits the declarations of terminals and non-terminals from the rest of the file.
- A set of rules, composed of:
 - a non-terminal
 - an arrow (`->`)
 - a sequence (possibly empty) of terminals and non-terminals.

For example, the rule `TERM -> FACTOR FACTORS` means that the non-terminal `TERM` can be recognized by first recognizing the non-terminal `FACTOR`, and then the non-terminal `FACTORS`. Another example, the rule `FACTORS ->` means that the non-terminal `FACTORS` can be recognized by an empty sequence of symbols.

5.1.2 ALPAGA Options

The ALPAGA program kindly responds to the `--help` option:

```
$ alpaga --help
Usage:
  -g Input grammar file (.g)
  -t Where to output tables (.html)
  -pml Where to output the parser code (.ml)
  -help Display this list of options
  --help Display this list of options
$ alpaga/alpaga -g grammar-file.g -t table.html -pml parser.ml
```

This will read the grammar file `grammar-file.g` and generate the parser code in `parser.ml`. Another output from the generator is a `table.html` file. This file, which you can open in a web browser, will show your grammar in a clickable format, the Null, First, and Follow tables needed to create the parser, and finally the resulting LL table. In particular, the cells of the table will be displayed with a red background if a conflict has been detected. If this is the case, you will need to rewrite your grammar to remove ambiguities.

Note: Your compiler expects to find the parser generated in the file `src/generated_parser.ml`. Make sure not to upset it. (Or rather: do not run ALPAGA manually; the Makefile will take care of that for you very well.)

The file `expr_grammar_action.g` contains the beginning of a grammar with a rule that includes an action (code within braces). Ignore this piece of code for now; we will return to it in the following questions.

5.1.2.1 Informal grammar of E.

In order to guide you in writing the grammar of the E language, here is an informal grammar of the language.

- A program consists of a list of function declarations (for the moment, only one function, which must be called `main`).
- A function declaration consists of the name of the function followed, in parentheses, by a list of arguments separated by commas, and finally followed by the body of the function.
- The body of a function is composed of statements. Statements can be:
 - a block of instructions inside curly braces
 - a conditional of the form `if (c) { i1 } else { i2 }`, where `c` is an expression, and `i1` and `i2` are sequences of instructions. The `else` part is optional.
 - a loop of the form `while (c) i`, where `c` is an expression, and `i` is a statement.
 - an instruction `return e`; where `e` is an expression.
 - an instruction `print (e)`; where `e` is an expression.
- The expressions are similar to those in C, where one is limited to the operators `==`, `!=`, `<`, `>`, `<=`, `>=`, `+`, `-`, `*`, `/`, `%`, as well as the unary operator `-`. The *basic* expressions are integers and variable identifiers. Sub-expressions can also be parenthesized. You can draw inspiration from the operator precedence in C¹.

Question 5.11

Complete the file `expr_grammar_action.g` with the grammar for language E, in the format expected by ALPAGA. Your parser should consume the lexemes but not produce an AST (you haven't done anything for that yet).

As you write your grammar, run `make` at the root of your project and examine the [_build/default/grammar.html](#) file to ensure that you are not introducing conflicts.

This simple parser will allow you to ensure that your grammar is correct, regardless of the actions you will write afterwards. Thus, on a syntactically correct program, your parser should succeed silently. For a syntactically incorrect program, you should get a syntax error.

Your parser should succeed on all `.e` files in the `ecomp-tests` directory, except for the `syntax_error*.e` files. More precisely, the table in [.test-results/results.html](#) should contain an E generation error, and no longer a syntax error.

At this stage, you therefore know how to recognize syntactically valid programs, but you do not construct an abstract syntax tree. Let us now focus on this problem.

ALPAGA allows specifying, for each rule of the grammar, an **action**, *i.e.* an OCaml expression that constructs *something* for this rule. By default, when no action is explicitly specified, the

¹https://en.cppreference.com/w/c/language/operator_precedence

action used is `()` (the constant of type `unit`). To specify an action, it is enough to add, at the end of the line corresponding to a rule, OCaml code between braces.

For example, consider the following grammar with actions:

```
tokens SYM_EOF SYM_IDENTIFIER<string> SYM_INTEGER<int> SYM_PLUS SYM_ASTERISK
non-terminals S EXPR TERM FACTOR INTEGER IDENTIFIER
non-terminals TERMS FACTORS
{
  let resolve_associativity term terms = ...
}
rules
S -> EXPR SYM_EOF { $1 }
IDENTIFIER -> SYM_IDENTIFIER { StringLeaf($1) }
INTEGER -> SYM_INTEGER { IntLeaf($1) }
EXPR -> TERM TERMS { resolve_associativity $1 $2 }
TERM -> FACTOR FACTORS { resolve_associativity $1 $2 }
TERMS -> SYM_PLUS TERM TERMS { (Tadd, $2) :: $3 }
TERMS -> { [] }
FACTOR -> IDENTIFIER { $1 }
FACTOR -> INTEGER { $1 }
FACTORS -> SYM_ASTERISK FACTOR FACTORS { (Tmul,$2) :: $3 }
FACTORS -> { [] }
```

Some remarks on this grammar:

- A block of code, enclosed in braces, was inserted just before the line ‘`rules`’. This block of code will be inserted into the generated parser code, at the very beginning of the file. Functions can therefore be defined there, modules can be imported (`open Symbols`), which will be usable in the grammar actions.
- The rules for the non-terminals `IDENTIFIER` and `INTEGER` build leaves of the abstract syntax tree. The type of the trees is defined in `src/ast.ml`.
- Special variables `$1`, `$2`, ... can be used in actions. The variable `$i` corresponds to the OCaml term returned by the *i*-th element of the rule. Specifically, in the rule:

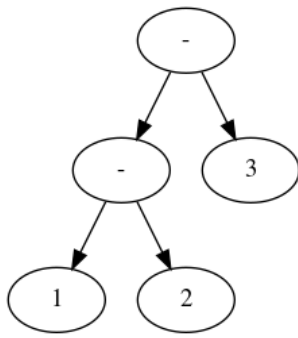
```
TERMS -> SYM_PLUS TERM TERMS { (Tadd, $2) :: $3 }
```

The variable `$2` corresponds to the result returned by `TERM` and the variable `$3` corresponds to the result returned by `TERMS`.

- On terminals, the variable `$i` returns the string that was used to recognize this terminal. This is used in particular for terminals `SYM_IDENTIFIER` and `SYM_INTEGER` which are respectively of type `string` and `int`.
- In rule `EXPR -> TERM TERMS { resolve_associativity $1 $2 }`, a function is called on the elements produced by the non-terminals `TERM` and `TERMS`. By looking a bit at the other rules, one understands that a call to `resolve_associativity` could be made with the parameters:

```
resolve_associativity (IntLeaf(1)) [(Tsub,IntLeaf 2); (Tsub, IntLeaf 3)]
```

which corresponds to the syntactic analysis of the sentence `1 - 2 - 3`. The purpose of the function `resolve_associativity` is to construct the following tree:



```

Node (Tsub, [
  Node (Tsub, [IntLeaf 1, IntLeaf 2]);
  IntLeaf 3
])
  
```

Question 5.12 — Action!

Add actions to your grammar in order to build an AST similar to the one shown above. Refer to the file `src/ast.ml` which documents the type of abstract syntax trees.

You will specifically need to write the body of the function `resolve_associativity`, a skeleton of which is found in the file `expr_grammar_action.g`.

When you have written the actions, run `make test` from the root of your project so that:

- ALPAGA regenerates the parser in the file `src/generated_parser.ml` ;
- the entire compiler must be recompiled (wow!);
- the tests be restarted.

After that, in the table of [.test-results/results.html](#), the page accessible from each link corresponding to a file name should contain an abstract syntax tree (a graphical representation of the tree that you will have built). It should look like this (for the test program `ecomp-tests/basic/gcd.e`):

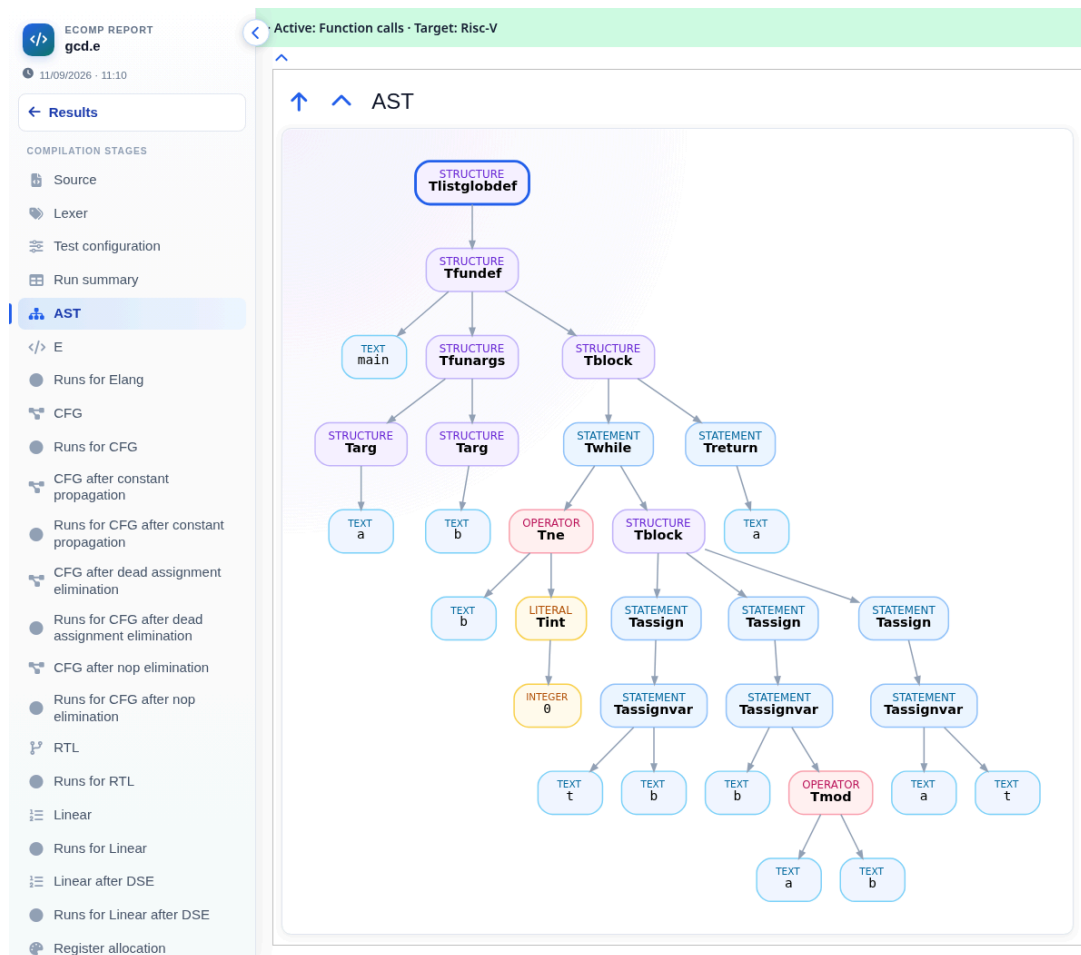


Figure 13: The AST section of the generated report.

6 TP3: Generation and interpretation of E programs

Expected test coverage

E generation and the E interpreter should pass `ecomp-tests/basic/`.

From the AST, you will now have to generate an E program and write an interpreter for these programs. The OCaml representation of E programs is defined in the file `elang.ml`, described in class.

The file `elang_print.ml` contains some functions for displaying expressions, statements, and E programs. Take a look at it to debug your programs.

6.1 Generation of E

The goal is to generate an E program from an AST. The following functions are to be completed in the file `src/elang_gen.ml`.

Question 6.13

Write the function `make_eexpr_of_ast (a: tree) : expr res` which transforms a subtree `a` into an E expression. This function returns an error if the tree does not meet the expected format.

Question 6.14

Write the function `make_einstr_of_ast (a: tree) : instr res` which transforms a subtree `a` into an E instruction. This function returns an error if the tree does not meet the expected format.

Question 6.15

Complete the function `make_fundef_of_ast (a: tree) : (string * efun) res` which converts a sub-tree `a` into an E function definition. This function returns an error if the tree does not meet the expected format.

The function `make_eprog_of_ast (a: tree) : eprog res` is already written. It transforms a subtree `a` into an E program. This function returns an error if the tree does not match the expected format.

6.2 Interpretation of E

One way to check that your AST construction and transformation into E is correct is to interpret your programs, *i.e.*, execute them. To do this, you will write a function that takes an E program and a list of arguments, and returns the value returned by that program.

You will need a **program state**: a data structure that stores the value of each variable throughout the execution of your program. The file `prog.ml` contains, in particular, the type `'a state`.

```
type 'a state = {
  env: (string, 'a) Hashtbl.t;
  mem: Mem.t
}

let init_state memsize =
{
  mem = Mem.init memsize;
  env = Hashtbl.create 17;
}
```

We will come back to the ‘memory’ component later, when we need it. For now, let’s focus on the environment. It is a hash table where the keys are variable names and the values are of type `'a`. At first, we will instantiate `'a` with `int`. Later, we can enrich it with the type of variables.

We will therefore use the functions `Hashtbl.replace : ('a, 'b) Hashtbl.t -> 'a -> 'b -> unit` and `Hashtbl.find_opt : ('a, 'b) Hashtbl.t -> 'a -> 'b option` to write and read in the environment.

Question 6.16 — Parameter passing.

In the function `eval_eprog : (oc: formatter) (ep: eprog) (memsize: int) (params: int list): int option res`, build the initial state of the program with the list of arguments passed as parameters. Note that the list contains only the values; the names of the parameters are in the program itself (in `f.funargs`).

Save the generated E artifact, then run it with `ecomp-run` and the parameters you want to pass:

```
$ ./ecomp -f my-file.e -e-save /tmp/my-file.ecomp-e
$ ./ecomp-run -e-load /tmp/my-file.ecomp-e -e-run -json - -- 4 12
```

The explicit `-json -` asks `ecomp-run` to print its result on standard output. The per-file report generated by `make test` presents the same result in a more readable form.

Note: ignore the superfluous command line parameters (for example, if your program expects 2 parameters and you provide 4, ignore the last two).

Let's now move on to the evaluation of expressions!

Question 6.17

Write the bodies of the functions `eval_unop : unop -> int -> int` and `eval_binop : binop -> int -> int -> int`, which take an operator (unary or binary) and their integer parameters, and return the result.

Question 6.18

Write the function `eval_eexpr : int state -> expr -> int res` which evaluates an expression `e`, given a state `s` (of type `int state`).

Question 6.19

Write the function `eval_einstr (oc: formatter) (st: int state) (ins: instr): (int option * int state) res` that executes the instruction `ins` with the state `st`.

If the instruction to be executed contains a return, a return value can be returned: it is the first component of the result (`int option`).

The state can change, particularly during assignments: the second component of the result (`int state`) gives the new state.

Ultimately, all this can fail (reading an uninitialized variable), hence the type `res`.

The function `eval_einstr` is already called in the function `eval_eprog`, so you should have finished this part of the lab.

7 TP4: Liveness analysis and dead code elimination

Expected test coverage

Liveness analysis and dead-assignment elimination should preserve the complete `ecomp-tests/basic/` suite, including `useless-assigns.e`.

The goal of this session is to implement an optimization for our compiler: the elimination of dead assignments. This optimization is based on a preliminary analysis of the program's code, which will be facilitated by the use of an appropriate intermediate language: CFG. The CFG language uses the same expressions as the E language, and a subset of the same instructions. The main differences are as follows:

- a program is a control flow graph (hence the name of the language: CFG for Control Flow Graph);
- control-flow instructions (IF and WHILE) are encoded directly in the structure of the graph.

The CFG language is described in the file `src/cfg.ml`.

In particular, it defines the expression type `expr`. These are the same expressions as in E. However, when extending your compiler, the E expressions and the CFG expressions may evolve differently, which is why we preferred to duplicate this type from the start.

A CFG function (type `cfg_fun`) is composed of a list of arguments (`cfgfunargs`), a hash table of nodes whose keys are the node identifiers (integers) and whose values are `cfg_node`, and the entry point of a CFG is the identifier of a particular node of the CFG. The instruction stored at each CFG node has type `cfg_node`:

- `Cassign(v,e,s)` is an assignment `v := e`. `s` is the identifier of the successor node, *i.e.* to which node one must jump next.
- `Creturn (e)` is a function return, returning the value of the expression `e`. This instruction has no successor.
- `Cprint(e,s)` displays the value of the expression `e`, then continues execution at the node `s`.
- `Ccmp(e, s1, s2)` evaluates the expression `e`. If its value is true (not zero), then the execution continues at node `s1`; otherwise at node `s2`.
- `Cnop s`: performs no operation, then jumps to node `s`.

The functions `succs` and `preds` respectively give the successors and predecessors of a node in a CFG.

We provide the transformation pass that generates a CFG program from an E program (`cfg_gen.ml`), an interpreter for the CFG language (`cfg_run.ml`), and a viewer for CFG programs (`cfg_print.ml`).

To inspect a CFG, run `make test DIR=basic/useless-assigns.e`, open [.test-results/basic/useless-assigns.e.html](#), and select one of the CFG sections in the report sidebar. The report renders the graph and the effect of each optimization for you.

To invoke the CFG interpreter manually, first save the artifact and then load it with `ecomp-run`:

```
$ ./ecomp -f my-file.e -cfg-save /tmp/my-file.ecomp-cfg
$ ./ecomp-run -cfg-load /tmp/my-file.ecomp-cfg -cfg-run -json - - 4 12
```

The runner output is a JSON object. The `"runstep": "CFG"` entry contains a `"retval"` attribute (the return value), an `"output"` attribute (text printed by the program), and an `"error"` attribute.

When you launch `make test`, the HTML page generated for each test file will contain a graphical representation of the program CFG.

The optimization we are interested in is **dead-assignment elimination**. This optimization allows removing from the program assignments `v := e` such that the value of `v` is never read after this assignment. It is easy to understand that in this case, this assignment can be removed without changing the behavior of the program.²

To determine which assignments can be eliminated, we will proceed with a liveness analysis of the variables.

7.1 Liveness analysis

This section takes place in the file `cfg_liveness.ml`. The goal of this analysis is to obtain, for each node of the program, the set of variables that are *live* before and after this node. In fact, it is enough for us to calculate the set of live variables **before** each node; if needed, we can calculate from this the live variables after each node.

To store all the live variables before each node, a hash table indexed by integers (the identifiers of the CFG nodes) will be used: `(int, string Set.t) Hashtbl.t`.

The analysis will be carried out by calculating the fixed point of the data flow equations seen in class, until a stable solution is found (*i.e.* until no more live variables are added at any program point).

Each iteration of the analysis will update the analysis state, *i.e.* the mapping between node identifiers and the list of live variables before that node. For each node, we will start by calculating the set of live variables after that node: this is the union of the live variables before each of its successors. From this set, we will calculate the set of live variables before that node: the variables read become live and the variables written become dead.

Question 7.20

Write a function `vars_in_expr : expr -> string Set.t` that returns the set of variables used by an expression.

We provide you, in the file `cfg.ml`, the functions:

- `succs : (int, cfg_node) Hashtbl.t -> int -> int Set.t`; and
- `preds : (int, cfg_node) Hashtbl.t -> int -> int Set.t`;

which return respectively the successors and predecessors of a node in a CFG.

²Attention, in languages richer than language E, such as C for example, this is only valid if the expression `e` has no side effects.

Question 7.21

Write a function `live_after_node: (int, cfg_node) Hashtbl.t -> int -> (int, string Set.t) Hashtbl.t -> string Set.t`. More precisely, `live_after_node cfg n lives` computes the set of variables alive after executing the node `n` in the graph `cfg`, given the set of variables alive before each node provided by the table `lives`.

In other words, this computes out in the data flow equations.

Question 7.22

Write a function `live_cfg_node : cfg_node -> string Set.t -> string Set.t`. For example, `live_cfg_node node live_after` must return the set of variables live before the node `node`, where `live_after` is the set of variables live after this node.

In other words, this computes in in the data flow equations.

Question 7.23

Use the previous functions to write a function `live_cfg_nodes (cfg: (int, cfg_node) Hashtbl.t) (lives : (int, string Set.t) Hashtbl.t) : bool` that performs one iteration of the fixed-point calculation on the CFG `cfg` starting from the state `lives`.

This function updates the `lives` table and returns a boolean indicating whether changes occurred on at least one node.

Question 7.24

Finally write the function `live_cfg_fun : cfg_fun -> (int, string Set.t) Hashtbl.t`. This function computes the set of live variables before each node of the CFG corresponding to a given function.

7.2 Removal of dead code

This part takes place in the file `cfg_dead_assign.ml`. We will now use the result of our analysis to optimize our program. We will traverse the control flow graph and for each node corresponding to an assignment (`Cassign`), if the variable assigned is not live after the assignment, we will remove this assignment (more precisely, transform the node into a new node of type `Cnop`).

Question 7.25

Write a function `dead_assign_elimination_fun (f: cfg_fun) : cfg_fun` which, given a CFG function, computes the liveness of variables and eliminates dead assignments.

In some cases, the transformed program can still be simplified by applying this same transformation. Indeed, some variables could only be live because they were used in an assignment

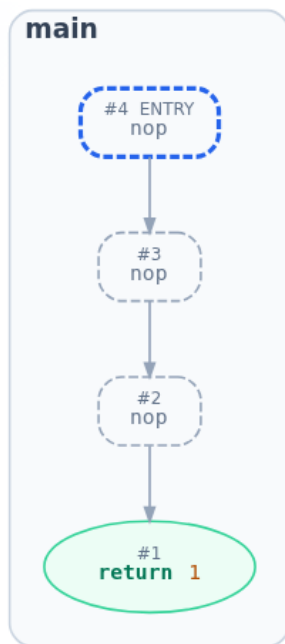
involving a variable that was itself dead. This is the case, for example, with the example `useless-assigns.e` present in `ecomp-tests/basic/`.

Question 7.26

Modify your code to apply the transformation as much as necessary.

7.3 Bonus: elimination of NOPs

In the previous section, some nodes were transformed into `Cnop`, that is to say no op or no operation. We are now interested in eliminating these NOPs.



For example, the CFG for the program `ecomp-tests/basic/useless-assigns.e` is shown on the left.

A certain number of functions to be completed are present in the file `src/cfg_nop_elim.ml`.

On the example left of this text:

- `nop_transitions cfg = [(4,3); (3,2); (2,1)]`
- `follow 4 _ _ = 1`
- `follow 3 _ _ = 1`
- `follow 2 _ _ = 1`
- `follow 1 _ _ = 1`
- `nop_transitions_closed cfg = [(4,1); (3,1); (2,1); (1,1)]`

After applying the optimization, the CFG should look like the single node shown in the left hand side of this page.

8 TP5: Generation of RTL programs

Expected test coverage

RTL generation and interpretation should pass `ecomp-tests/basic/`.

The objective of this lab session is to generate RTL (*Register Transfer Language*) programs. As explained in [Section 2](#), RTL is a language in which the program is a control flow graph, like in CFG, but where expressions, unlike in CFG, are broken down into elementary operations on

registers. There is no limit to the number of registers a RTL program uses. In RTL, the notion of variable is lost: all data is in (pseudo-) registers.

Why choose such an intermediate language? It brings us closer to the assembly code ultimately generated and is therefore *lower-level* than the previous languages. It is a common choice in well-known compilers: the GCC compiler uses a language that is also called RTL and shares a number of characteristics with our RTL language, Clang's intermediate representation IR also resembles it, and CompCert also uses an RTL language.

The description of the RTL language is in `src/rtl.ml`. It includes the register type `reg` (a synonym for `int`).

The RTL instructions (type `rtl_instr`) are:

- Binary operation `RBinop(b, rd, rs1, rs2)`: we perform the binary operation `b` on the registers *source* `rs1` and `rs2`, and we store the result in the register *destination* `rd`.
- Unary operation `RUnop(u, rd, rs)`: the unary operation `u` is performed on register *source* `rs`, and the result is stored in register *destination* `rd`.
- Operation `Rconst(r, i)`: we store the constant `i` in the register `r`.
- Operation `Rmov(rd, rs)`: we copy the value contained in register *source* `rs` into *destination* register `rd`.
- Label operation `Rlabel l`: a *label* is defined at this point in the code. A jump can target it later, but this operation does nothing.

Note : labels will only be used starting from the Linear language. For now, we do not produce labels in RTL code, and the targets of the jump instructions `Rbranch` and `Rjmp` are identifiers of the nodes in the CFG.

- Branch operation `Rbranch(cmp, rs1, rs2, l)`: we evaluate the comparison `cmp` (of type `rtl_cmp` defined above) on the registers `rs1` and `rs2`. If the comparison evaluates to true (non-zero), we jump to *label* `l`; otherwise, we continue the evaluation normally.
- Unconditional jump operation `Rjmp l`: we jump to *label* `l`.
- Operation `Rprint r`: displays the contents of register `r`.
- Operation `Return r`: we return the value contained in the register `r`, and we stop the execution of the current function.

8.1 Generation of RTL programs

Here is the attack plan for transforming CFG programs into RTL programs.

1. First of all, we need a way to generate new register names. The functions we are going to write will therefore take as a parameter a pair `(next_reg, var2reg)`, where:
 - `next_reg` is the number of the next available register (not yet allocated to a variable, or used as an intermediate register);
 - `var2reg`: `(string * int) list` is an association list that contains a pair `(v, r)` if the RTL register `r` is associated with the CFG variable whose name is `v`.
2. Equipped with these parameters, we will need to transform CFG expressions and instructions into RTL operations.

The work to be done is in the file `src/rtl_gen.ml`.

The function `find_var (next_reg, var2reg) v`, written for you, returns a triplet `(r, next_reg', var2reg')` where `r` corresponds to the register corresponding to the variable `v`, and `next_reg'` and `var2reg'` are the updated input parameters.

Let's now move on to the compilation of expressions. The compilation of a CFG expression will produce a list of RTL operations that will have the effect of computing the expression in question and storing its value in a register. The function `rtl_instrs_of_cfg_expr (next_reg, var2reg) e` performs this transformation and must return a quadruple `(r, l, next_reg', var2reg')` such that:

- the result of the CFG expression `e` is stored in the register `r` ;
- the list of RTL instructions corresponding to the calculation of this result is `l` ;
- `next_reg'` and `var2reg'` are the variables `next_reg` and `var2reg` passed as parameters, possibly updated.

For example, to compile the expression `a + 2 * b`, assuming that `next_reg` is 0 and `var2reg` is initially empty:

- we start by compiling the sub-expression `a`. We will then call the function `find_var`, which returns the triplet `(0, 1, [("a", 0)])`.

The result of the compilation of this first subexpression will give the quadruplet `(0, [], 1, [("a", 0)])`. (No operation is required.)

- we then compile the sub-expression `2 * b`.

First of all, the compilation of the sub-expression `2` will return the quadruple: `(1, [Rconst(1, 2)], 2, [("a", 0)])`.

Then, the compilation of the sub-expression `b` will return the quadruple: `(2, [], 3, [("a", 0); ("b", 2)])`.

Finally, we will obtain for the sub-expression `2 * b`:

`(3, [Rconst(1,2); Rbinop(Emul, 3, 1, 2)], 4, [("a", 0); ("b", 2)])`.

- We combine these two sub-results: we need a new register to store the result of the expression as an integer:

`(4, [Rconst(1,2); Rbinop(Emul, 3, 1, 2); Rbinop(Eadd, 4, 1, 3)], 5, [("a", 0); ("b", 2)])`

Question 8.27

Write the function `rtl_instrs_of_cfg_expr (next_reg, var2reg) e`.

The expressions are now compiled. Let us move on to the CFG nodes. They are transformed into RTL nodes with the same identifiers. Rather than containing a CFG instruction, they contain a list of RTL operations.

Question 8.28

Write the function `rtl_instrs_of_cfg_node (next_reg, var2reg) (c: cfg_node)` which returns, given a CFG node, a triple `(l, next_reg', var2reg')` where `l` is the list of RTL operations corresponding to a CFG node `c` and `next_reg'` and `var2reg'` have the same meaning as before.

You will note the presence of the function `rtl_cmp_of_cfg_expr`: we leave it to you to interpret what it does and how to use it... Probably for branching operations?

The following function, `rtl_instrs_of_cfg_fun`, has been written for you and only uses the previous functions to build an RTL function. Take a look at it to understand what is happening there.

You now have RTL programs!

A RTL display and interpreter are provided to you. The simplest way to inspect the generated RTL is to run the test and open its per-file report:

```
$ make test DIR=basic/gcd.e
```

Open [.test-results/basic/gcd.e.html](#) and select the RTL section in the left column. To run RTL manually, first ask `ecom` to save the compiled artifact, then load it with `ecom-run`:

```
$ ./ecom -f ../ecom-tests/basic/gcd.e -rtl-save /tmp/gcd.ecomp-rtl
$ ./ecom-run -rtl-load /tmp/gcd.ecomp-rtl -rtl-run -json - - 21 14
```

8.2 Linearization of RTL programs

RTL programs are control flow graphs. The next phase is the linearization of programs, that is, transforming the graph into a Linear language program (defined in the file `src/linear.ml`), which is just a list of RTL instructions, including explicit jumps.

A naive linearization is implemented in the file `src/linear_gen.ml`. This linearization is naive because the different blocks of RTL instructions are placed one after the other without consideration of the control flow graph, and may therefore use too many jumps.

So you have a linearization that works, but which could be improved. Here's how:

- we could arrange the nodes of the original control flow graph according to a topological order, that is to say, a depth-first traversal of the graph.
- if two blocks of instructions are ordered one right after the other, a jump can be avoided. In other words, there is no need for the sequence of instructions `[Rjmp l; Rlabel l]` (jump to the next instruction).
- after the previous transformation, we can eliminate the labels to which no instruction jumps.

Since this section is optional, few details are provided, and it is up to you to explore how to implement the different optimizations presented above.

9 TP6: Register Allocation

Expected test coverage

Register allocation should preserve `ecomp-tests/basic/`; run it with both the default allocator and `-clever-regalloc` when that option is available.

The purpose of this lab session is to carry out register allocation, which will allow the production of assembly code that can be assembled and executed by a RISC-V processor.

Register allocation consists of specifying, for each RTL (or Linear) pseudo-register, which hardware location will store it. Concretely, we associate each RTL pseudo-register with either a hardware register or a location on the stack.

We provide you with a very simple ‘register allocator’ that allocates all RTL pseudo-registers on the stack and therefore does not use any hardware registers. It is certainly inefficient since this will generate a large number of memory reads/writes, but it has the merit of working.

You therefore already have a compiler that generates executable RISC-V programs!

```
$ ./ecomp -f ../ecomp-tests/basic/gcd.e
$ ../ecomp-tests/basic/gcd.exe 54 24

6
$ ./ecomp -f ../ecomp-tests/basic/prime.e
$ ../ecomp-tests/basic/prime.exe 54
54
2
3
3
3

0
```

We know how to calculate the GCD of two integers and decompose an integer into prime factors!

Moreover, the HTML report generated for each source file (for example [.test-results/basic/gcd.e.html](#)) contains, in the section **LTL**, the allocation of each RTL pseudo-register. For example, for the test `ecomp-tests/basic/gcd.e`, we have the following allocation:

```
// In function main
// LinReg 1 allocated to a1
// LinReg 0 allocated to a0
// LinReg 4 allocated to stk(-32)
// LinReg 3 allocated to stk(-24)
// LinReg 2 allocated to stk(-16)
```

You will notice that some pseudo-registers (1 and 0) are allocated in real machine registers (`a1` and `a0`) while the others (2, 3, and 4) are allocated on the stack, at locations -16, -24 and -32.

The goal of the remainder of this session is therefore to write a smarter register allocator, which will make better use of the registers available in RISC-V.

9.1 Crash course: RISC-V assembler

RISC-V is an open instruction set architecture (ISA). It follows the RISC (*Reduced Instruction Set Computer*) design, with relatively few instructions and many registers. In comparison, x86 is a CISC (*Complex Instruction Set Computer*) architecture with many instructions and relatively few registers. We present here a subset of the RISC-V instructions—the ones needed to write our compiler. Note that we provide you with the code for generating the RISC-V assembler (in `src/riscv.ml`). The descriptions that follow will therefore be useful for your knowledge, as well as for understanding the code provided to you when you need to extend it.

9.1.1 RISC-V Registers

RISC-V exists in 32-bit, 64-bit, and even 128-bit versions. We are going to write code for RISC-V 32 and 64; most of the code can be shared. In these architectures, RISC-V provides access to 32 registers of 32 or 64 bits each (depending on the architecture), and here is the list:

Number	Name	Comment	Number	Name	Comment
x0	zero	always zero	x16	a6	arguments
x1	ra	return address	x17	a7	
x2	sp	stack pointer	x18	s2	
x3	gp	global pointer	x19	s3	
x4	tp	thread pointer	x20	s4	
x5	t0	temporaries	x21	s5	callee-save registers
x6	t1		x22	s6	
x7	t2		x23	s7	
x8	s0/fp	frame pointer	x24	s8	
x9	s1	callee-save register	x25	s9	
x10	a0	argument / return value	x26	s10	temporaries
x11	a1	arguments	x27	s11	
x12	a2		x28	t3	
x13	a3		x29	t4	
x14	a4		x30	t5	
x15	a5		x31	t6	

The zero register always contains the value 0.

The register `ra` is used to store the return address during a function call.³

The `sp` register contains the stack pointer (similar to `esp` in x86).

The register `gp` is *global pointer*. It contains the address of the memory area where global variables are stored. We will not use it in our compiler.

The register `tp` is *thread pointer*. It contains the address of the memory area specific to the current thread. We will not use it in our compiler.

The registers `t0` to `t6` are temporary registers that do not have any specific purpose.

In this compiler, the register `s0` is used as `fp`, a *frame pointer* containing the address of the start of the current function's stack frame. It is the analogue of the `ebp` register in x86. The RISC-V ABI does not require a dedicated frame pointer; this is a convention adopted here.

The registers `s1` to `s11` are callee-save registers. A function that modifies one of them must restore its original value before returning.

³Note that this is only a convention and that the return address could be stored in any register.

The registers `a0` to `a7` are used to pass arguments during a function call (first argument in `a0`, second in `a1`, etc.). Additionally, the return value of functions is passed in the register `a0`.

9.1.1.1 Caller-save and callee-save registers.

The registers `ra`, `t0`-`t6`, and `a0`-`a7` are *caller-save*, meaning that it is the responsibility of the calling function (the *caller*) to save these registers before making a function call if their value is important.

The registers `sp`, `s0`-`s11` are *callee-save*, that is to say that it is the responsibility of the called function (the *callee*) to save these registers if the called function modifies them. We can therefore assume, when calling a function, that the value of these registers will be the same after the call.

9.1.2 Instructions.

Unlike x86, where instructions accept multiple addressing modes for operands (registers, immediate values, memory locations), RISC-V instructions generally have a single addressing mode.

Most of the instructions are of the ‘3-address’ type and expect registers as operands. Two main schemes emerge:

- `op rd, rs1, rs2` where `op` is an operation (`add`, `mul`, `div`, `remu` (for *remainder unsigned*, a.k.a. modulo)...), `rd` is the destination register, and `rs1` and `rs2` are the source registers.
- `op rd, rs` where `op` is an operation (`neg`, ...), `rd` is the destination register, and `rs` is the source register.

To compile conditionals (for example `r1 <- r2 <= r3`), the following additional instructions are needed:

- `slt rd, rs1, rs2`: `rd` is 1 if `rs1 < rs2`, 0 otherwise
- `seqz rd, rs`: `rd` is 1 if `rs` is zero, 0 otherwise
- `snez rd, rs`: `rd` is 0 if `rs` is zero, 1 otherwise

9.1.2.1 The branching instructions.

- Unconditional jump: `j <label>`
- Function call: `jal ra, <label>`
- Jump if two registers are equal: `beq rs1, rs2, <label>`
- Jump if `rs1 < rs2` (in unsigned or not): `blt[u] rs1, rs2, <label>`
- Jump if `rs1 >= rs2` (in unsigned or not): `bge[u] rs1, rs2, <label>`
- Jump if `rs != 0`: `bnez rs, <label>`

9.1.2.2 Access to memory

- Read from memory to a register: `ld rd, ofs(rs)`. This instruction reads the memory at address `rs + ofs` and stores the result in register `rd`.

`ld` means *load double-word*, i.e. 64 bits, and is used on RV64. On RV32, use `lw` for a 32-bit *load word*; for 16 bits use `lh` for *load half-word*, and for 8 bits `lb` for *load byte*.

- Write the value of a register into memory: `sd rs, ofs(rd)`. This instruction writes the value of the register `rs` into memory at address `rd + ofs`.

`sd` means *store double-word*, i.e. 64 bits, and is used on RV64. On RV32, use `sw` for a 32-bit *store word*; for 16 bits use `sh` for *store half-word* and for 8 bits `sb` for *store byte*.

9.1.2.3 Immediate values

To write a constant into a register: `li rd, imm`, where `rd` is the destination register and `imm` is the constant that we want to store.

9.1.3 Provided code

We provide you with a “naive” register allocator, in the file `src/regalloc.ml`. More precisely, the register allocator `regalloc_on_stack_fun` takes as input a function `linear_fun` and returns a pair `((reg, loc) Hashtbl.t * int)`. The first element of the pair is a hash table that associates each pseudo-register with a location (*location* in English) of type `loc` defined as follows:

```
type loc = Reg of int | Stk of int
```

A location is therefore either a physical register of the target processor (here the RISC-V processor) or a slot on the stack. Such a slot is an offset relative to the `fp` register (*frame pointer* or stack frame). Note that since on RISC-V (as on x86 as well) the top of the stack (the `sp` register) decreases towards smaller addresses when data is pushed onto it, the offset between a local variable spilled onto the stack and the stack frame register `fp` is always a negative integer, as shown in the figure:

2nd argument
1st argument
saved ra
saved s0
s0 (frame pointer)
local variables

The provided allocator works in a very simple way by going through all the instructions of a function in Linear language to determine the pseudo-registers it uses. For this, the functions `gen_live` and `kill_live`, which are provided to you in the file `src/linear_liveness.ml`, are used, the purpose of which is precisely to calculate the liveness of pseudo-registers in Linear language. You are not required to code this liveness analysis yourself since you did it already for the CFG liveness in a previous lab session. By merging all the pseudo-registers used by a function, all that remains is to iterate over the list of these registers to allocate to each a slot (a negative integer) that decreases from 1 for each of them (since for the moment all variables are of integer type with a length equal to that of the underlying architecture).

9.2 Register allocation

Although we now have a complete compiler (in the sense that we generate executable files), we can make a large number of improvements to our compiler. In this part, we are going to write a register allocator in order to take advantage of the multitude of registers available in RISC-V; and minimize the number of instructions that access memory.

We are going to write an algorithm based on graph coloring, as seen in class. For this, it is necessary first to carry out a liveness analysis of the pseudo-registers used in the Linear language. As explained earlier, this analysis is provided to you (file `src/linear_liveness.ml`).

When you run a test on an example, the **Linear** section of its per-file report shows the program together with the live-before and live-after sets computed for each instruction:

```
$ make test DIR=basic/gcd.e
```

Open [.test-results/basic/gcd.e.html](#) and select **Linear** in the left column.

From this analysis, we are going to construct an interference graph: the vertices are the different pseudo-registers used in the program, and there is an edge between two pseudo-registers if they are live at the same time. Intuitively, two pseudo-registers that are live at the same time cannot be assigned to the same machine register. The register allocation problem then becomes a graph coloring problem: we want to assign a color to each vertex so that two adjacent vertices in the graph are assigned different colors.

9.2.1 Register allocation by graph coloring

Let's now move on to the construction of the interference graph and its coloring. This takes place in the file `src/regalloc.ml`.

9.2.1.1 Construction of the interference graph.

The construction of the interference graph will be carried out by the function `build_interference_graph (live_out : (int, reg Set.t) Hashtbl.t) : (reg, reg Set.t) Hashtbl.t` which returns the interference graph from a liveness analysis. The liveness analysis is provided in the form of a hash table that associates each instruction of the code in the Linear language (identified by an integer: its position in the program) with a set of registers live at the exit of that instruction. This function returns an interference graph in the form of a hash table that associates each register with the registers with which it interferes and is therefore adjacent in the graph.

It is up to you to build the graph, based on the result of liveness. You must add edges between two vertices if at least at one program point, the pseudo-registers associated with these two vertices are alive at the same time.

Question 9.29

Write a function `add_interf (rig : (reg, reg Set.t) Hashtbl.t) (x: reg) (y: reg) : unit` which updates the interference graph `rig` (for *register interference graph*) by adding a link between the vertices `x` and `y`.

Attention, given the representation chosen for the graph (in the form of adjacency lists), adding an edge involves adding `x` to the neighbors of `y` and `y` to the neighbors of `x`.

Question 9.30

Write a function `make_interf_live (rig: (reg, reg Set.t) Hashtbl.t) (live : (int, reg Set.t) Hashtbl.t) : unit` that builds the interference graph.

We provide you the function `build_interference_graph (live_out : (int, reg Set.t) Hashtbl.t) : (reg, reg Set.t)`

`Hashtbl.t` which ensures that the interference graph is correctly initialized with as many

vertices as there are pseudo-registers used in the Linear program. This function uses the functions you defined previously.

9.2.1.2 Coloring of the graph

The register allocation algorithm by graph coloring was initially proposed in a research paper by Chaitin *et al.* in 1981⁴.

Assume that we have N colors (machine registers). The algorithm proceeds in several phases:

1. We start by building a stack of pseudo-registers by indicating for each whether we will definitely be able to associate a color (a machine register) with it, or not (in which case it will have to be allocated on the stack – we say it is spilled).

The construction of this stack is carried out as follows:

As long as the rig graph is not empty:

- we are trying to find a vertex with strictly fewer than N neighbors in rig
 - If such a vertex r exists, let's remove it from the graph rig and add this register to the top of the stack indicating that we will know how to assign a color to it. In our code we put on the stack `NoSpill r`.
 - Otherwise, we will have to evict (*spiller*) a pseudo-register, that is to say choose a pseudo-register which will be allocated on the stack. We can choose any vertex, but a particular heuristic is to choose the vertex that has the most neighbors: this will have the effect of reducing the number of neighbors of many other registers and will make it easier later to find a vertex with fewer than N neighbors. We remove this vertex from the graph, and we add it to the top of the stack indicating that it will be evicted. In our code, we put on the stack `Spill r`.
1. Once this stack is built, we will indeed associate colors with each vertex marked `NoSpill` and stack locations with each vertex marked `Spill`.

Question 9.31

Write a function `remove_from_rig (rig : (reg, reg Set.t) Hashtbl.t) (v: reg) : unit` that removes a vertex v from the interference graph rig passed as a parameter.

Attention, given the representation chosen for the graph (in the form of adjacency lists), deleting a vertex requires going through all the elements of the hash table to remove any reference to that vertex.

Question 9.32

Write a function `pick_node_with_fewer_than_n_neighbors (rig: (reg, reg Set.t) Hashtbl.t) (n: int) : reg option` that returns `Some (r)`, with r a register from rig having fewer than n neighbors when possible, and `None` otherwise.

⁴Chaitin, Gregory J.; Auslander, Marc A.; Chandra, Ashok K.; Cocke, John; Hopkins, Martin E.; Markstein, Peter W. (1981). "Register allocation via coloring".

Question 9.33

Write a function `pick_spilling_candidate` (`rig : (reg, reg Set.t) Hashtbl.t`) : `reg option` that returns a vertex that will be evicted. Choose the vertex with the maximum number of neighbors. As with the previous function, return `Some (r)` when the graph is not empty or `None` otherwise.

You are provided with the type `regalloc_decision = Spill of reg | NoSpill of reg`

Question 9.34

Write the recursive function `make_stack` (`rig : (reg, reg Set.t) Hashtbl.t`) (`stack : regalloc_decision list`) (`ncolors: int`) : `regalloc_decision list` which, given an interference graph, a decision stack (already partially built and possibly empty), and a number of available registers, extends the stack passed as a parameter as described above.

Question 9.35

Write the function

```
allocate
  (allocation: (reg, loc) Hashtbl.t)
  (rig: (reg, reg Set.t) Hashtbl.t)
  (all_colors: int Set.t)
  (next_stack_slot: int)
  (decision: regalloc_decision) : int
```

which extends an allocation (possibly already partially built) from:

- `alloc`: a partial allocation;
- `rig`: the interference graph, useful for knowing the neighbors of a vertex;
- `all_colors`: a set of available physical registers (the colors);
- `next_stack_slot`: an integer (negative or zero as explained above) representing the next slot to use on the stack (if needed);
- `decision`: a decision (of type `regalloc_decision`) stored on the decision stack This function returns the next location on the stack that will need to be used in the future. This means that it can decrement the value `next_stack_slot` that was passed to it as a parameter.

The functions `make_stack` and `allocate` are called by the function

```
let regalloc_fun (f: linear_fun)
  (live_out: (int, reg Set.t) Hashtbl.t)
  (all_colors: int Set.t)
  : (reg, reg Set.t) Hashtbl.t      (* the interference graph *)
  * (reg, loc) Hashtbl.t          (* the allocation *)
  * int                          (* the next available stack slot *)
= ...
```

which is given to you already completed. This function performs register allocation for a function in a Linear program. It returns a triple containing:

1. the interference graph which can be visualized from its representation in file `.dot`.
2. a hash table describing an association between each pseudo-register of the program in Linear and a location;
3. the next free position on the stack. This function is itself called by the function `regalloc lp lives all_colors` which does the same work but on a complete Linear program (possibly composed of multiple functions).

Run the `gcd.e` test with the allocator you have just programmed:

```
$ make test DIR=basic/gcd.e COMPILER_OPTS=-clever-regalloc
```

Then open [.test-results/basic/gcd.e.html](#). The **Register allocation** section shows the interference graph and its coloring, while the **LTL** section shows the resulting allocation and generated program.

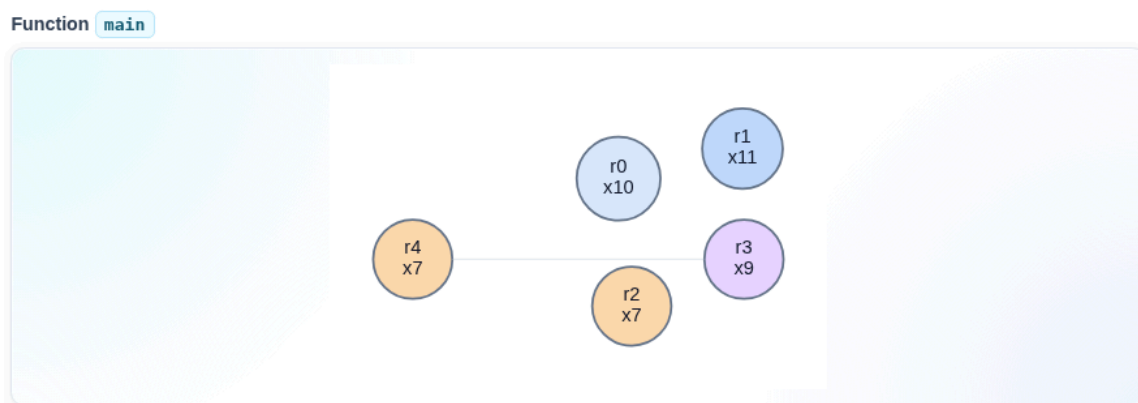


Figure 16: Register allocation in the generated per-file report.

We can see that the registers `r2` and `r4` are allocated in the same physical register (`t2`, according to the previous LTL dump).

The registers `r3` and `r4` are in different registers because they interfere with each other. Indeed, we had seen in the Linear dump, a few pages earlier, that these registers were alive together at a program point.

The arguments `r0` and `r1` are preallocated to the calling-convention registers `a0` and `a1`. The other colors are chosen by the allocator; for example, the report above assigns both `r2` and `r4` to `x7` (`t2`).

9.3 Testing

To run the tests, you have used the command `make test` since the beginning of these lab sessions. But you do not know all the power that is hidden behind this command!

By default, `make test` launches the Python script `ecomp-tests/test.py` on all the `*.e` files located in the `ecomp-tests/basic` directory. Also, by default, the “naive” register allocator is used.

You can change this default behavior by passing parameters to your `make` command.

To pass additional parameters to the compiler (*i.e.* to `./ecomp`), use the `COMPILER_OPTS` variable as follows:

```
# pass the -clever-regalloc option to the compiler
$ make test COMPILER_OPTS=-clever-regalloc
```

To run tests on a set of source files, these parameters are passed into the `DIR` variable as follows:

```
$ make test DIR="basic/prime.e basic/gcd.e"
```

You can obviously combine these two options:

```
$ make test COMPILER_OPTS=-clever-regalloc DIR="basic/prime.e basic/gcd.e"
```

10 Function calls

Expected test coverage

The untyped function-call extension should pass `ecomp-tests/funcall/`. Keep the earlier `ecomp-tests/basic/` suite passing as well.

At this point, the language accepted by your compiler is somewhat limited. In particular, we do not know how to make function calls. This is the next step to improve your compiler. You will then be able to compile programs that look like this one:

```
fib(n){
  if(n > 14){ return -1; }
  if(n <= 2){
    return 1;
  } else {
    return fib(n-1) + fib(n-2);
  }
}
main(n){
  return fib(n);
}
```

As you can see, this is an implementation of the Fibonacci sequence in a recursive and inefficient way (the same values are calculated multiple times...). To prevent the calculations from taking too long, we give up and return `-1` for an input greater than 14.

Note that the language is still untyped: all variables are integers. The above program, as well as ten other programs, is provided to you in the `funcall` directory.

To extend your compiler, you will need to modify your grammar, as well as each of the intermediate languages and the different compilation passes. Here is an overview of the changes to make:

10.1 The grammar

A statement can now be a function call (in addition to the existing cases of assignment, loops, ...):

```

x = 3;
y = 8;
f(x,y); // A function call used as a statement.
z = x + y;

```

A **expression** can also contain function calls:

```

x = 3;
y = 8;
z = 5 * f(x,y) + 16; // A function call used as an expression.

```

Expand your grammar to recognize these new constructions.

Also write the actions corresponding to these new constructions, in order to obtain an AST resembling this one:

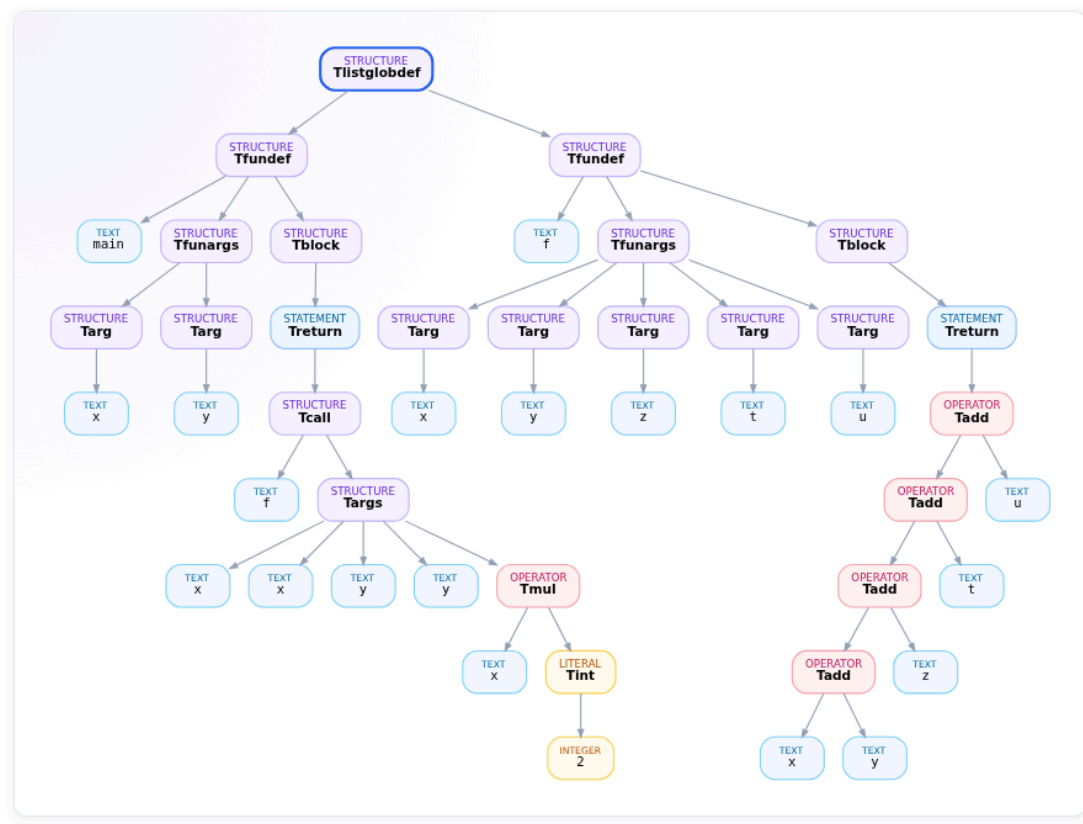


Figure 17: The AST section of the generated report for 5args.e.

The source program generating this AST is as follows (ecomp-tests/funcall/5args.e):

```

main(x, y){
    return f(x,x,y,y,x*2);
}
f(x, y, z, t, u){
    return x + y + z + t + u;
}

```

You can of course add to the file `src/ast.ml` the new AST constructors that seem necessary to you (`Tcall`, `Targs` ...).

10.1.0.1 Tips for modifying the grammar

It is important to check that your grammar does not contain any conflicts each time you modify it. To do this, there are two solutions:

1. When you run the make command, the grammar is recompiled and a warning message will be issued if a conflict is present.

Warning! There is a conflict in your grammar. Check the prediction table for more details.

2. As the warning message suggests, go to the prediction table to identify the conflict. By default, the table is generated in the file `_build/default/grammar.html` at the root of your project. In the last table of the file, a red square indicates a conflict. This means that the parser does not know which rule to apply to continue the analysis.

To resolve these conflicts, it is necessary, in particular, to ensure that no two rules can start with the same terminal. Therefore, these rules must be factored.

```
A -> X A { Node (Ta, [$1; $2])}
A -> X B { Node (Tb, [$1; $2])}
```

then becomes

```
A -> X A_or_B
A_or_B -> A
A_or_B -> B
```

However, writing the actions becomes tedious. We can get around this in different ways.

10.1.0.2 Solution 1: define types

In the header of the grammar (before the keyword rules and between the braces, where you must have already written the function `resolve_associativity`), you can define a type specific to your parser:

```
{
  type parser_choice =
    | ChoiceA of tree
    | ChoiceB of tree
}
...
A -> X A_or_B {
  match $2 with
  | ChoiceA t -> Node (Ta, [$1; t])
  | ChoiceB t -> Node (Tb, [$1; t])
}
A_or_B -> A { ChoiceA $1 }
A_or_B -> B { ChoiceB $1 }
```

- Good: it also works if we use `A_or_B` in multiple places in the grammar
- Bad: it's a bit heavy to define new types and to do a match ...

10.1.0.3 Solution 2: returning functions

The actions can be of any type, including functions!

We can rewrite the previous example as follows:

```

A -> X A_or_B { $2 $1 }
A_or_B -> A { fun x -> Node (Ta, [x; $1]) }
A_or_B -> B { fun x -> Node (Tb, [x; $1]) }

```

In the action of the first rule, we call the function returned by the non-terminal `A_or_B` with, as an argument, the element returned by the non-terminal `X`.

- Good: more elegant and shorter than the previous solution
- Bad: since we wrote all the logic of the action in `A_or_B`, we cannot (unless we pass other arguments) use this non-terminal in multiple places, in slightly different contexts.

10.2 The E language

Extend the types of expressions and E statements in the following way, in `src/elang.ml`.

```

type expr =
  ...
  | Ecall of string * expr list

type instr =
  ...
  | Icall of string * expr list

```

An expression `Ecall(fname, args)` represents a call to the function named `fname` with the arguments represented by the list of expressions `args`. The value of this expression is the return value of the function `fname`. If the function does not return a value, it is an error.

Similarly, a `Icall(fname, args)` instruction represents a call to the function named `fname` with the arguments represented by the list of expressions `args`. The return value of the function is ignored.

10.2.1 Generation of E

In the file `src/elang_gen.ml`, you now need to process the new types of nodes (`Tcall`, in the previous figure) introduced in the previous section, in order to produce `Ecall` expressions and `Icall` instructions.

10.2.2 E Interpreter

Now that an expression can contain function calls, evaluating an expression may require executing a function. To do this, you will need to write mutually recursive definitions, that is, the body of the function `eval_eexpr` may call the function `eval_efun`, which itself calls the function `eval_einstr`, which itself calls the function `eval_eexpr` ...

The OCaml syntax for writing mutually recursive definitions is as follows:

```

let rec eval_eexpr st e = ...

and eval_einstr ... = ...

and eval_efun ... = ...

```

You will probably need to add arguments to these functions, in particular `eval_eexpr`.

Note: so far, functions do not modify memory – the only side effects that can be observed are text displayed on the screen. Nevertheless, in order to prepare for the future, we will suppose

that memory can be modified and we will take extra care to propagate the effects of function calls onto the program state. More concretely, if you must evaluate a list of expressions `[e1; e2]` (for instance for the arguments of a function call), keep in mind that the evaluation of `e1` might modify the state and influence the evaluation of `e2`. The return type of the `eval_expr` function could have to be extended...

10.3 The CFG language

As with language E, a new type of expression is added `Ecall(fname, args)` and a new type of node `Ccall(fname, args, s)` where `s` is the successor of this node, as in `Cassign`.

The changes to be made in `src/cfg_run.ml` are similar to those you will have made in `src/elang_run.ml`.

Since your data types have changed, the compiler will complain that your match are not exhaustive. Satisfy the compiler by completing these match.

10.4 The RTL language

A `Rcall` of `reg option * string * reg list` operation is introduced.

The operation `Rcall(ord, fname, rargs)` means that the function `fname` is called with the arguments stored in the pseudo-registers `rargs`. If `ord = Some rd`, the result of the function will be stored in the register `rd`, otherwise, if `ord = None`, the result of the function will be ignored. If `ord = Some rd` but the function does not return a value, it is an error.

10.5 The LTL language

The greatest difficulty in handling function calls lies in the generation of assembly code (or LTL, it's almost the same thing).

Indeed, at this point, one must be able to call a function (via the instruction `Lcall (fname)`), already present in `src/ltl.ml`. But it is also, and above all, necessary to generate the assembly code that:

1. saves the so-called *caller-save* registers (*i.e.* those that must be saved by the caller): indeed, the called function will perhaps (surely) use these registers to perform a calculation, and their value in the calling function would be lost;
2. provides the arguments to the function;
3. makes the actual call (instruction `Lcall`);
4. retrieves the return value of the function (stored by convention in the `a0` register) to the expected location (as calculated by register allocation);
5. restores the value of the registers *caller-save*.

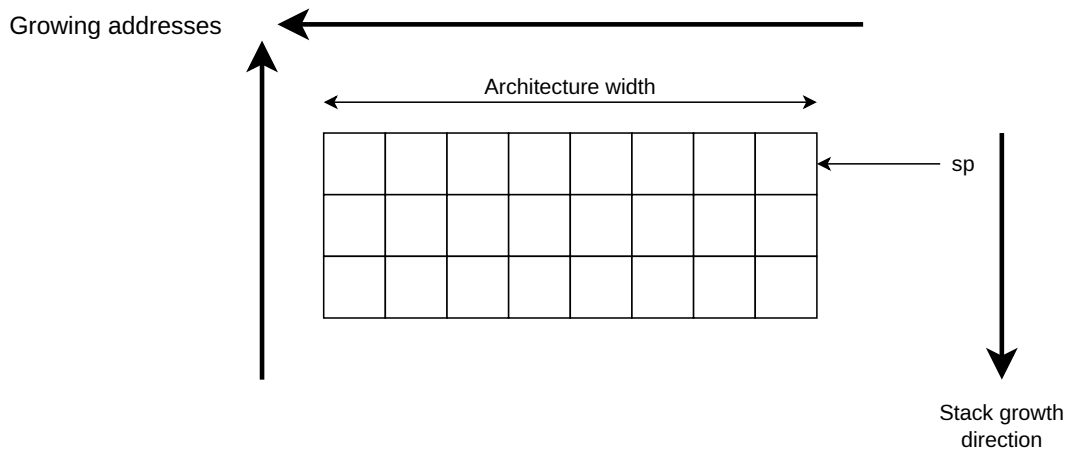


Figure 18: General organization of the stack

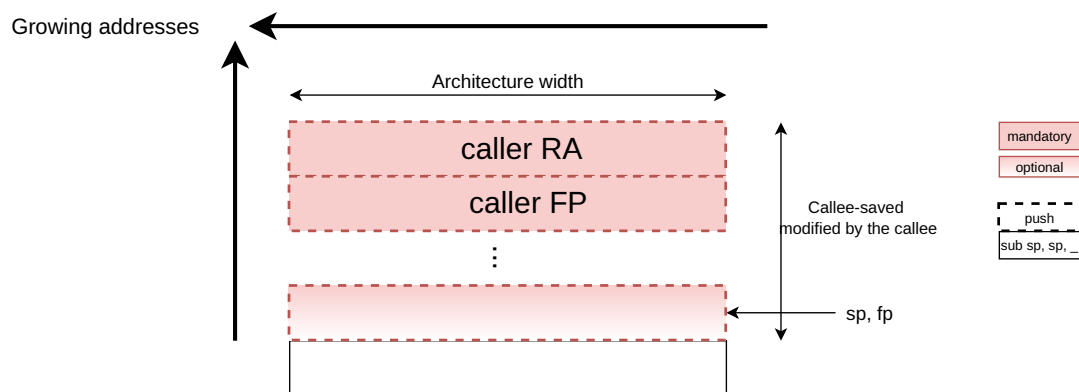


Figure 19: Management of registers saved by the callee

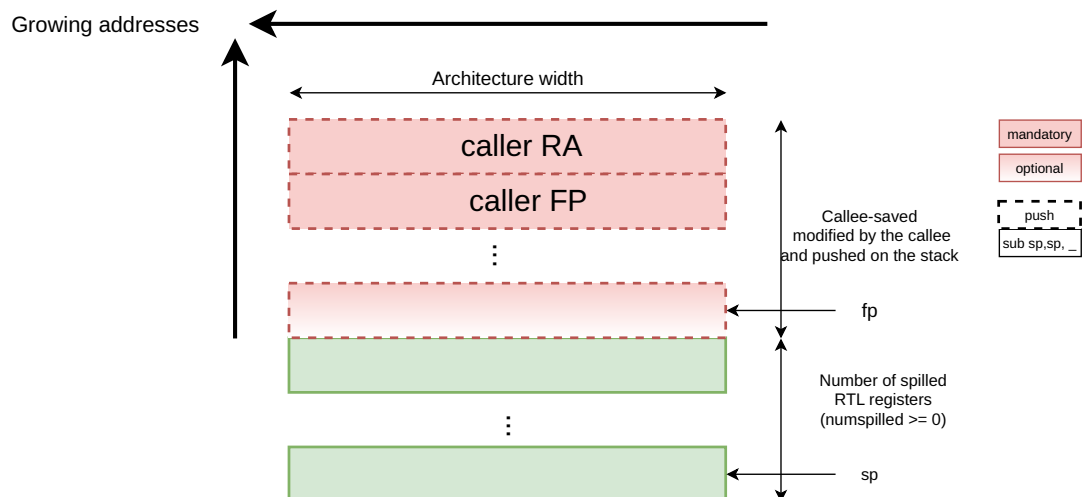


Figure 20: Management of RTL registers allocated on the stack by the register allocator

Stack management in LTL and RISC-V at function entry

The changes you need to make are located in the file `src/ltl_gen.ml` in the function `ltl_instrs_of_linear_instr`, where you need to handle the case where the Linear instruction you need to translate is `Rcall(rd, callee_fname, rargs)`.

10.5.1 Saving caller-save registers

First, we need to identify all the registers to save. A naive solution is to save all the *caller-save* registers, namely the `a0` to `a7` registers and the `t0` to `t6` registers. In your code, this would be written as `arg_register @ reg_tmp`.

A more economical solution would be to save only the *caller-save* registers that are live after the function call. This information is available via the `live_out` argument passed to the `ltrl_instrs_of_linear_instr` function. The set of registers to save is calculated by the `caller_save` function, defined just above the `ltrl_instrs_of_linear_instr` function. This function takes the following parameters:

- `live_out`: the result of liveness analysis on the outgoing edges;
- `allocation`: the register allocation for this function;
- `rargs`: the list of registers to pass as arguments.

The reason for going through the list of argument registers is as follows. Remember that the arguments of a function are passed in the registers `a0` to `a7`: the first argument in `a0`, the second in `a1`, etc. Suppose we have a Linear function call `f(r1, r2)`, and that `r1` is associated with the register `a1` and `r2` with the register `a0` (this is possible if `r1` and `r2` are arguments of the calling function, as illustrated by the test `ecompe-tests/funcall/argswap.e`).

So, during the parameter passing (the next step), we will want to write `a1` into `a0` and `a0` into `a1`. If we do naively:

```
mv a0, a1
mv a1, a0
```

So, it is obvious that the value contained in `a1` will not be the correct one; it will have been overwritten by the previous `mv`.

That is why the function `caller_save` anticipates this kind of problem, calculates all the arguments that will be overwritten (function `overwritten_args`) and includes them in the set of registers to save.

Now that the registers to be saved are identified, it is necessary to issue the instructions that save them. The function `save_caller_save` does this job, and takes the following parameters:

- `to_save`: `ltrl_reg list`: the set of variables to save (we just calculated it!);
- `ofs`: `int`: the number of the first location on the stack available to perform this save.

If we go back to the stack diagram from the previous section, and complete it.

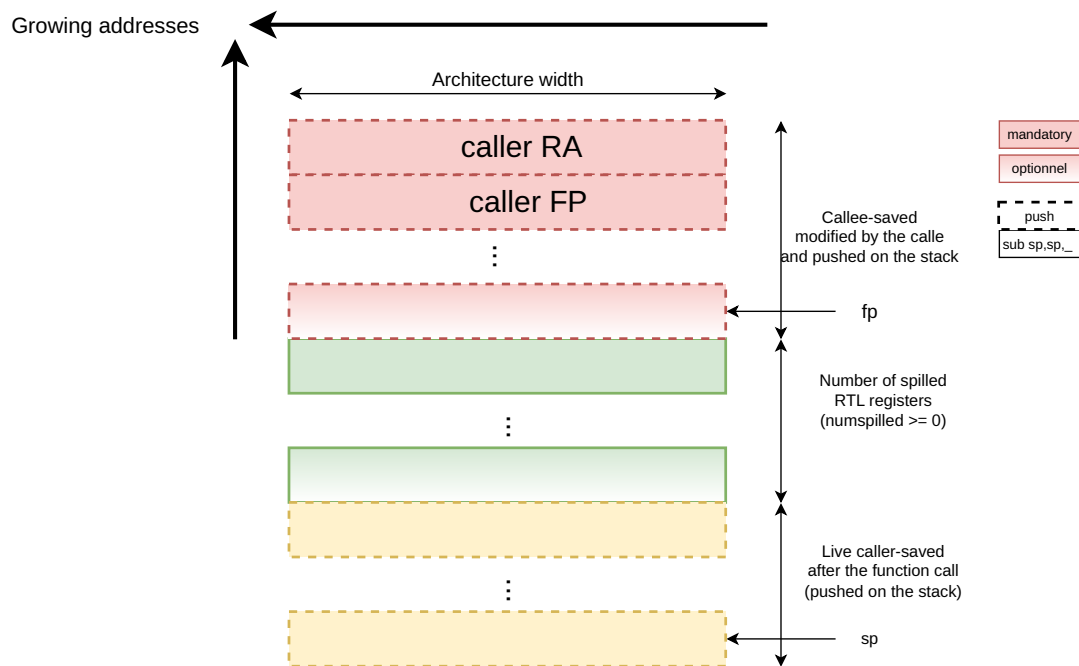


Figure 21: Backup of caller-save registers

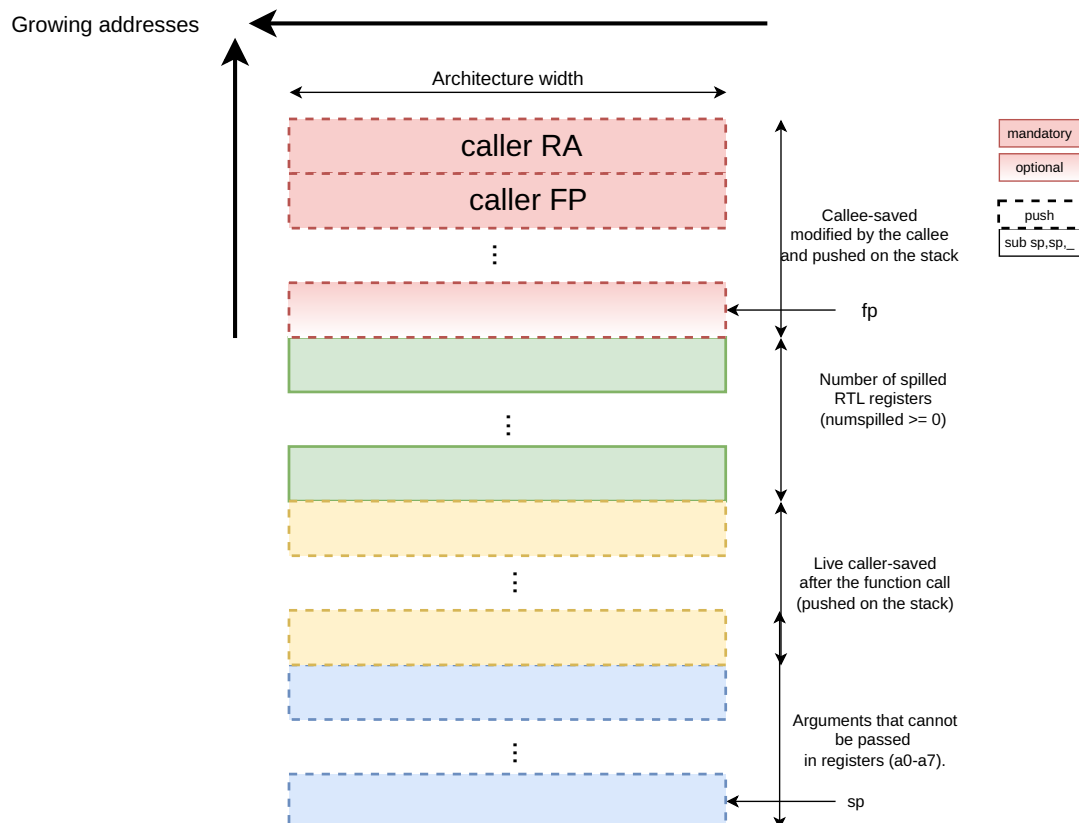


Figure 22: Passing of superfluous arguments

Stack management in LTL and RISC-V during a function call

The location planned for register backups is at the address pointed to by `s0`, minus the space reserved for spilled registers, minus the space reserved for the function's local variables.

We do not yet have local variables for a function that we store on the stack, so it is sufficient for us to have the number of spilled variables, which is available to us in the parameter `num_spilled` of the function `ltl_instrs_of_linear_instr`.

The function `save_caller_save` returns a triplet (`save_regs_instructions`, `arg_saved`, `ofs`), where:

- `save_regs_instructions`: `ltl_instr list` is the list of LTL instructions that handle saving the registers to `_save` onto the stack;
- `arg_saved`: (`ltl_reg * int`) `list` is an association list: a pair (`r`, `o`) in this list means that the LTL register `r` has been saved at the location `o` on the stack (`ofs` for the first, `ofs - 1` for the second, etc.)
- `ofs` is the new top location available on the stack, after saving the registers. We will need it for what follows.

After this list of instructions, a series of writes to addresses `s0 - X` has been issued. It is now necessary to position the stack pointer `sp` just after the saved registers: this is where the value `ofs` returned by the function `save_caller_save` is used.

10.5.2 Passing of parameters

Next comes the moment of passing the parameters. The function `pass_parameters`, aptly named, takes care of this task. It takes the following arguments:

- `rargs`: `reg list`: the list of RTL/Linear pseudo-registers that contain the arguments;
- `allocation`: register allocation;
- `arg_saved`: the result of `save_caller_save`, to know where the arguments are saved, if necessary. It can fail if certain `rargs` registers are not allocated in `allocation`. If it succeeds, it returns a pair (`parameter_passing_instructions`, `npush`), where:
- `parameter_passing_instructions` is a list of LTL instructions, which perform parameter passing;
- `npush` is the number of arguments that we had to pass on the stack. (Most of the time, the arguments will go into the `a0-7` registers, but for functions with more than 8 arguments (like in `ecomp-tests/funcall/lots_of_args.e` for example), the stack will be used.)

If necessary, this function performs “pushes” on the stack, that is, it decrements the `sp` register and writes to the obtained address.

10.5.3 Call of the function

Here, it is very simple, you just need to issue the instruction `LTL LCall callee_fname`, where `callee_fname` is the name of the called function.

10.5.4 Unstacking of arguments

We restore the stack pointer `sp` just before the arguments, *i.e.* we replace the stack pointer `npush` slots higher in the stack.

10.5.5 Return value

The function return value is stored in `a0`; copy it to the desired location (`rd`), if necessary.

10.5.6 Restoring caller-save registers

Finally, restore the values of the registers that were saved. The function `restore_caller_save` calculates the list of instructions that perform this task; it takes as an argument an association list like the one returned by `save_caller_save`.

Be careful: if the location corresponding to the register in which the return value must be stored is a physical register, the value of this register must not be restored!

10.6 Built-in functions

Now that you are able to handle function calls, you will notice that the `print` instruction is actually a special case of a function call. We can therefore remove the `print` instruction from our compiler to turn it into a simple function call.

Here are the places where you will need to make changes:

- the lexer: the token `SYM_PRINT` no longer has a reason to exist.
- grammar: a special case is no longer necessary.
- each of the intermediate languages: no need for a dedicated display instruction!
- the interpreters of each of these intermediate languages: we will use the *built-in* mechanism!

The file `src/builtins.ml` contains a number of predefined functions *built-in* for you. In particular, the function `do_builtin oc mem fname vargs` provides the semantics of the function call *built-in* whose name is `fname` with the arguments given in `vargs`. The parameter `oc` is a *output channel* as you have already handled so far. The parameter `mem`, of type `Mem.t`, is a memory, which you have not needed yet (but very soon, promised!). In interpreters of different languages, you can pass the memory associated with the state `st` like this: `st.mem`.

This function has as return type `int option`

`res :`

- `OK (Some i)` means that the function evaluated without error and returns the integer value `i`.
- `OK None` means that the function evaluated without error and does not return a value.
- `Error msg` means that the function evaluated with the error `msg`.

10.7 Debugging low-level programs

From now on, you may need to debug the programs generated by your compiler in order to find implementation errors present in your compiler. We have added a new section ([Annexe A](#)) to the handout that details how to debug programs at the LTL and even RISC-V level.

11 Typing

Expected test coverage

Typed programs should pass `ecomp-tests/type_basic/` and `ecomp-tests/type_funcall/`. Character support is covered by `ecomp-tests/char/`.

The purpose of this extension of your compiler is to add types to your language. Initially, we will restrict ourselves to the types `int`, `char`, and `void`. In later extensions, we will add pointers, arrays, and structures.

A new set of test files is offered to you: the directories `ecomp-tests/type_basic/`, `ecomp-tests/type_funcall/`, and `ecomp-tests/char/`. Take a look at some of these files before reading further, to become familiar with the new language you will have to compile.

Types are a high-level concept: that is to say, they only impact the first few intermediate languages. In our case, you will only have to modify your `expr_grammar_action.g` grammar and the files related to the E language.

To start, add the following definitions to the file `src/prog.ml`:

```
type typ =
  | Tint
  | Tchar
  | Tvoid

let string_of_typ t =
  match t with
  | Tint -> "int"
  | Tchar -> "char"
  | Tvoid -> "void"
```

We thus define a type `typ` which corresponds to the different types that we will support initially. We also define a function to display these types.

Here is an overview of the steps you will need to go through. These are deliberately not very detailed to give free rein to your imagination!

1. Modify your grammar to support:
 - the types in function signatures: return type and argument types
 - local variable declarations, with or without an initialization (*e.g.* `int x;` or `int x = 3 * z;`)
 - the literal characters: `char c = 'A';`
2. Add the necessary tag definitions to `src/ast.ml`.
3. Add a constructor `Echar of char` to the expressions of E (`src/elang.ml`).
4. Modify the type of functions `efun` in `src/elang.ml` so that:
 - `funargs` should now be a list of `string * typ`, to retain the type of the arguments;
 - a new field `funvartyp: (string, typ) Hashtbl.t` registers the type of each local variable ;
 - a new field `funrettyp: typ` gives the return type of the function.
5. In `src/elang.ml`, write a function

```
type_expr
  (typ_var : (string, typ) Hashtbl.t)
  (typ_fun : (string, typ list * typ) Hashtbl.t)
  (e: expr) : typ res
```

that calculates the type of an expression.

Parameters:

- `typ_var` gives the type of each local variable of the current function.

- `typ_fun` gives the signature (type of the arguments + return type) of each function already processed in the program.
- `e` is the expression to type.

The function returns an error, particularly if a variable or a function appearing in the expression does not have a known type.

6. Still in `src/elang.ml`, add the parameters `typ_var` and `typ_fun` to the functions `make_eexpr_of_ast` and `make_einstr_of_ast`. Check (with a call to `type_expr`) that the constructed expressions have a type before returning them in `make_eexpr_of_ast`.

In `make_einstr_of_ast`, check that during an assignment `v = e`, the variable `v` and the expression `e` have compatible types. (`int` and `char` are compatible, `int` and `void` are not). (Write a separate function to test the compatibility of 2 types.)

Still in `make_einstr_of_ast`, update `typ_var` when you encounter a variable declaration. Check along the way that a variable of type `void` is not declared.

During function calls (whether as a `Ecall` expression or a `Icall` instruction), check that the arguments are of the expected type (compare with the signature that you can obtain via `typ_fun`).

7. In the function `make_fundef_of_ast`, initialize a `typ_var` table, then update the `typ_fun` table that you will need to receive as an argument.
8. In the function `make_eprog_of_ast`, initialize the table `fun_typ` by inserting into it in particular the type of three useful functions *built-in*:

```
let fun_typ = Hashtbl.create (List.length l) in
  Hashtbl.replace fun_typ "print" ([Tint], Tvoid);
  Hashtbl.replace fun_typ "print_int" ([Tint], Tvoid);
  Hashtbl.replace fun_typ "print_char" ([Tchar], Tvoid);
  ...
```

9. A function must be declared before another function can call it. (Typing should fail otherwise.)

How to manage mutually recursive functions? Look at the files `ecomp-tests/type_funcall/mutrec.e` and `ecomp-tests/type_funcall/mutrec-hotstadter.e`. They contain function declarations. Make sure to handle these files properly. (Evaluating the file `ecomp-tests/type_funcall/mutrec-hotstadter.e` takes some time ...)

12 Pointers

Expected test coverage

Pointer support should pass `ecomp-tests/ptr/`, while retaining all earlier suite coverage.

We are now going to add **pointers** to our language. In particular, look at the tests in the `ecomp-tests/ptr/` directory. These programs introduce two new constructions: the `&` operator, which represents the address of a variable, and the `*` dereferencing operator.

Thus, the following program, run with the parameters $x = 14$ and $y = 12$, should return the value 26:

```
void f(int* z, int x, int y){
    *z = x + y;
}
int main(int x, int y){
    int z = 3;
    f(&z, x, y);
    return z;
}
```

Indeed, the **address** of z is passed to the function f , along with the **values** of x and y . This function writes to the address contained in z , the sum of x and y . The function main ends up returning the value of z , which should contain 26.

These new operators bring about the following changes in our compiler:

- the grammar must recognize these new operators (by the way, the lexical analyzer must also recognize the new token $\&$ – why not call it `SYM_AMPERSAND` ?)
- the types (in `src/prog.ml`) are enhanced with a new constructor `Tptr ty`: a pointer to elements of type ty .
- we add to the E language (file `src/elang.ml`):
 - an expression `Eaddrof`
 - of `expr` to represent the address-of expression $\&e$. At this stage, e must be an addressable expression: a variable or a dereference.
 - an expression `Eload` of `expr` to symbolize the expression $*e$
 - an instruction `Istore` of `expr * expr` to symbolize the instruction $*e = v$;

In addition, a function (type `efun`) contains two new fields:

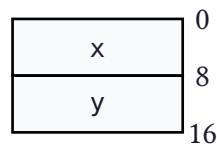
- a field `funvarinmem : (string, int) Hashtbl.t` which associates each variable name that must reside in memory with an offset in the stack block associated with each activation of this function
- a field `funstksz : int` which gives the size of the stack block of this function.

These last elements deserve a bit of explanation. Let's suppose you have the following C code:

```
int f(){
    int x = 3;
    int y = 4;
    int z = 5;
    z = g(&y, &x);
    return z;
}
```

In this example, we use the operator $\&$ on the variables x and y : these variables therefore need to be allocated in memory, more precisely on the stack since they are local variables to the function f . On the other hand, the variable z can remain, as all variables were until now, *temporaries* that are not allocated in memory.

We will therefore have for this example a stack block of 16 bytes (assuming that the `ints` are encoded on 8 bytes).



Here, the table `funvarinmem` will associate the variable `x` with offset (or *offset*) 0 and the variable `y` with offset 8. The field `funstksz` will be 16.

12.1 Generation of E

In previous sessions, you had to add a function `type_expr` to `src/elang_gen.ml`. Make sure to update the typing regarding pointers. Note that `p + i`, `i + p`, and `p - i` have the same type as `p` when `p` is a pointer and `i` is an integer (or a character, by the way). Given 2 pointers `p` and `q` of the same type, we can also compare these pointers and obtain an integer. All other unary and binary operations are undefined.

As seen previously, some variables will be allocated on the stack and others in temporaries. The criterion is as follows: if the address of a variable is taken in the program (using the `&` operator), then that variable will be allocated on the stack. (Later, non-scalar variables, *i.e.* arrays or structures, will be added to the stack.)

Write a function `addr_taken_expr (e: expr) :`

`string Set.t` that collects the set of variables whose address is taken in an expression `e`. Do the same for instructions with a function `addr_taken_instr (i: instr) : string Set.t`.

In the function `make_fundef_of_ast`, initialize a hash table `funvarinmem` that associates each variable allocated on the stack with an *offset* in the stack block associated with this function. To do this, you will need to define a function `size_type (t: typ) : int res` which returns the memory size occupied by a variable of type `t`. Define this function in `src/prog.ml`.

12.1.1 E Interpreter

The different functions of the E interpreter take an additional argument: a stack pointer `sp: int` which gives the starting address of the stack block in memory.

This stack pointer will be particularly useful for the expressions `Eaddr of` and `Eload`, as well as for the instructions `Istore`.

The memory is accessible from the state `st` via `st.mem`. The relevant memory operations are as follows:

- `Mem.write_bytes (m: Mem.t) (addr: int) (bytes: int list) : unit res`. This function writes into the memory `m` at the address `addr` the list of bytes `bytes`. Returns `OK ()` if everything went well, `Error msg` otherwise (access out of memory bounds).

This function is used in conjunction with the function `split_bytes n v` which splits an integer `v` into `n` pieces, each one byte long.

- `Mem.read_bytes_as_int (m: Mem.t) (addr: int) (sz: int) : int res`. This function reads from memory `m` at addresses `[addr; addr + sz[` and returns the integer represented by these bytes.

For the evaluation of binary operators, be careful with additions and subtractions on pointers. Let a pointer `p` of type `Tptr t` and an integer `i`:

- the expression `p + i` is actually evaluated in `p + i * sizeof(t)`

- the expression $p - i$ is actually evaluated in $p - i * \text{sizeof}(t)$
- the expression $i + p$ is actually evaluated in $i * \text{sizeof}(t) + p$

Also pay attention to the expressions `Evar v` and to the instructions `Iassign(v,e)`: the variable may be in memory and you need to access it correctly!

Also note that for the moment the operator `&` can only be applied to variables, and possibly to expressions `Eload e`.

To know how many bytes to read or write (expressions `Eload addr` and instructions `Istore(addr, v)`), refer to the type of the expression `addr` and to the size of the pointed type.

In the function `eval_efun`, reserve the necessary space on the stack by adjusting `sp` as needed.

You will need typing, so the function `type_expr` defined in `src/elang_gen.ml`. Don't forget to add a `open Elang_gen` at the top of your file. You will also need a table `typ_fun` that you can initialize in `eval_eprog` and propagate through the different functions. For the table `typ_var`, you should find what you need in the fields of the functions (type `efun`).

12.2 Generation of CFG

In the CFG language, we discard type information. We add, in the file `src/cfg.ml`:

- an expression `Estk of int`, which represents an address in the stack block of the current function;
- an expression `Eload of expr * int`. `Eload(e, sz)` represents a memory read at the address represented by `e`, over `sz` bytes
- an instruction `Cstore of expr * expr * int * int`. `Cstore(addr, v, sz, succ)` represents a memory write at the address represented by the expression `addr`, the value being written is that of the expression `v` over `sz` bytes. The successor of this instruction is `succ`, as for the instruction `Cassign` for example.
- a field `cfgfunstksz: int` of the type of CFG functions `cfg_fun`.

The compilation of E expressions and statements into CFG expressions and nodes mimics the E interpreter you just wrote. Therefore, you will need to perform the same type checks, adjust pointer operations, transform the `Evar v` and the assignments `Iassign(v,e)` into stack reads and writes when necessary.

12.3 CFG Interpreter

Similarly to the E language, the CFG language will need a stack pointer `sp` to evaluate our new expressions and instructions.

12.4 RTL

Add the operations to `src/rtl.ml`:

- `Rstk of reg * int`: `Rstk(rd, i)` writes in the register `rd` the address located at an offset `i` from the beginning of the stack block.
- `Rload of reg * reg * int`: `Rload(rd, rs, sz)` stores in the register `rd` the value at `sz` bytes read at the address contained in `rs`
- `Rstore of reg * reg * int`: `Rstore(rd, rs, sz)` writes the value contained in `rs` to `sz` bytes at the address contained in `rd`.

We also add a field `rtlfunstksz: int` to the functions `rtl_fun`.

Adapt the interpreter as seen before.

12.5 Linear

Since Linear shares all RTL operations, it is enough to add in `src/linear.ml` a field `linearfunstksz` to the type of functions.

Adapt the interpreter as seen before.

12.6 LTL

The function `ltl_instrs_of_linear_instr` takes a new argument `numlocals : int`: this is the number of slots on the stack that need to be reserved for local variables. This number will be calculated in the function `ltl_fun_of_linear_fun` based on the size `linearfunstksz` of the Linear function. Note that you must convert a number of bytes (`linearfunstksz`) to a number of slots (of size `!Archi.wordsize`, which is 8 in 64-bit or 4 in 32-bit).

Propagate this value wherever it seems necessary to you to comply with the diagrams of the stack diagrams above. Also make sure to properly compile the instructions `Rstk`, `Rload`, and `Rstore`.

In the function `ltl_fun_of_linear_fun`, adapt the *prologue* of the function to reserve space for the variables stored on the stack.

13 Structures

Expected test coverage

Structure definitions, field access, and structure pointers should pass `ecomp-tests/structs/`, while retaining the preceding suites.

We are now interested in **structures**. We want to be able to compile programs like the following:

```
struct mastruct {
    int x;
    int y;
};

int main(){
    struct mastruct S;
    S.x = 12;
    S.y = 3;
    return (S.x + S.y);
}
```

This test is given in `ecomp-tests/structs/struct.e`.

Here is an overview of the changes to be made.

13.1 The grammar

Modify the grammar to allow structure type definitions, as in lines 1 to 4 of the test above. A file then becomes a list of global definitions, each definition being either a function definition or a structure type definition.

Also allow functions to take structure arguments and local structure-variable declarations.

13.2 The E language

Add to the language E:

- an expression `Egetfield` of `expr * string : Egetfield(e, f)` represents an access to the field `f` of the structure whose address is represented by the expression `e`
- an instruction `Isetfield` of `expr * string * expr : Isetfield(s, fieldname, e)` represents writing the value of `e` into the field `fieldname` of the structure whose address is represented by the expression `s`.

Also, the type `eprog` becomes:

```
type eprog = efun prog * (string, (string * typ) list) Hashtbl.t
```

The second component of the pair is a hash table that provides, for each type of structure, the list of the structure's fields with their types. This will be useful to us, particularly for knowing the layout of the fields within the structures.

13.3 The file `src/prog.ml`

In `src/prog.ml`, add a new constructor to `typ` for the structure types: `Tstruct` of `string` (the type parameter `string` is the name of the structure).

Adapt the function `size_type` to calculate the size of the structures. Add to it as an argument a hash table like the one described above. For example, the size of the structure `mastruct` in the previous example is 16 (2 fields of size 8).

Also add the following two functions:

- ```
let rec field_offset
 (structs: (string, (string * typ) list) Hashtbl.t)
 (s: string) (f: string) : int res
```

A call to `field_offset structs s f` returns the offset of the field `f` in a structure of type `s`. For example, `field_offset structs "mastruct" "x"` will return `OK 0` and `field_offset structs "mastruct" "y"` will return `OK 8`. An incorrect call `field_offset structs "mastruct" "z"` will return `Error "No such field 'z'"`, or a similar error message in the language of your choice.

- ```
let rec field_type
  (structs: (string, (string * typ) list) Hashtbl.t)
  (s: string) (f: string) : typ res
```

A call to `field_type structs s f` returns the type of the field `f` in a structure of type `s`. For example, `field_type structs "mastruct" "x"` and `field_type structs "mastruct" "y"` will return `OK Tint`.

13.4 Generation of E

In `src/elang_gen.ml`, add an argument `typ_struct: (string, (string * typ) list) Hashtbl.t` to the functions `type_expr`, `make_eexpr_of_ast`, `make_einstr_of_ast`.

Modify the function `make_fundef_of_ast` so that it also handles structure type definitions. Its task will then be to update the table `typ_struct` when a structure definition is encountered.

During the construction of the `funvarinmem` set of local variables that must go into memory, add, in addition to the variables whose address is taken, the variables of type `struct`. Indeed, these must be allocated in memory. More generally, all variables that are not *scalar*, that is, of a *simple* type (integer, pointer, character), will be placed in memory.

It will also be necessary, obviously, to modify the functions `type_expr` and `make_eexpr_of_ast` to type and generate E expressions `Egetfield`, and the function `make_einstr_of_ast` to generate instructions `Isetfield`. Be sure to give as the first parameter of `Egetfield` and `Isetfield` the address of the structure: possibly introduce a `Eaddrof` ...

13.5 E Interpreter

In `src/elang_run.ml`, also add a parameter `typ_struct` where it seems necessary, of the same type as before. You will instantiate this parameter with the table generated in `src/elang_gen.ml`.

This will allow you to evaluate read and write access to the fields of a structure. Check, in the interpreter, that when evaluating `Egetfield(e, f)` or `Isetfield(s, fieldname, e)`, the receiver expression (`e` or `s`, respectively) is indeed of type `Tptr (Tstruct structname)`. The functions `field_offset` and `field_type` defined above in `src/prog.ml` should be useful to you.

For the evaluation of `Egetfield(e, f)`, if the field is of *scalar* type (integer, pointer, character), read the value contained in memory. Otherwise, if it is a structure, simply return the address of this field.

13.6 Generation of CFG

In `src/cfg_gen.ml`, also add a parameter `typ_struct`, again of the same type as before, again, where it seems relevant to you. You will instantiate this parameter with the table generated in `src/elang_gen.ml`.

We do not add specific expressions or instructions to structures in the CFG language. We will simply transform accesses to the fields of structures into simple memory accesses.

Thus, `S.y` will be replaced by `*(&S + 8)`, for example. The address of `S` will also have been replaced by a location on the stack: this will therefore actually give `*(stk(0) + 8)` or something similar.

Once this transformation is completed, there is no need to modify the rest of the compiler, it should normally already know how to handle memory reads and writes.

14 Arrays

Expected test coverage

Array declarations, indexing, and array/pointer compatibility should pass `ecomp-tests/array/`, while retaining the preceding suites.

We are now adding arrays!

Here is an example of a program that uses arrays (`ecomp-tests/array/array.e`):

```
int main(){
  int t[10];
  t[0] = 5;
  t[1] = 3 + t[0];
  return t[1];
}
```

We declare an array of 10 integers, initialize the first element to 5, the second to the value of the first plus 3, then return the value of this second element. The expected result, unsurprisingly, is 8.

14.1 The grammar

Extend the grammar to be able to declare variables of type array', and to access them for reading and writing (`t[i]`).

14.2 Generation and interpreter of E

Add a type in `src/prog.ml` that represents arrays: `Ttab of typ * int`. `Ttab(ty, n)` represents an array of `n` elements, each of type `ty`.

It should be noted that the types `Ttab(ty, n)` and `Tptr ty` are compatible, that is, one can provide a parameter of type `Ttab(ty, n)` to a function that expects an argument of type `Tptr ty`. Therefore, the evaluation of expressions will be modified, particularly so that additions and subtractions behave with arrays as they do with pointers.

Transform array accesses into simple memory accesses. Thus, the access `t[i]` is transformed into `*(t + i)`. There is no need here to multiply `i` by the size of the elements pointed to by `t`, this transformation is normally already done in the evaluation of addition between pointers and integers.

In the same way as for structures, when evaluating a `Eload (e)`, if the type pointed to by `e` is a scalar, its value in memory will be returned, but if it is a structure or array type, only its address will be returned.

14.3 Generation of CFG

As with the evaluation of E, arrays must be treated as pointers during additions and subtractions. That's all that needs to be changed!

15 Global variables

Expected test coverage

Global definitions, initialization, and access should pass `ecomp-tests/globals/`, while retaining the preceding suites.

We are now tackling global variables. Indeed, until now, all the data we could manipulate were variables local to a function. Here is an example of a program using global variables (`ecomp-tests/globals/globarray.e`):

```

int max = 20;                                0
int objs[20];                                0
int main(){                                  1
    int i = 0;                                3
    int sum = 0;                              6
    while (i < max){                          10
        objs[i] = sum;                        15
        sum = sum + i;                       21
        i = i + 1;                           28
    }                                          36
    i = 0;                                    45
    while (i < max){                          55
        print(objs[i]);                      66
        i = i + 1;                           78
    }                                          91
    return objs[max-1];                      105
}                                              120

```

Listing 1: `ecomp-tests/globals/globarray.e`

Listing 2: Expected output

Two global variables are defined there: an integer `max` and `objs`, an array of 20 integers.

We store in each cell i of the array the sum of the integers from 1 to i , then we display each of the 20 cells of the array. The expected output is given above.

15.1 The grammar

Extend the grammar to allow the definition of global variables, with or without initialization. Be careful not to create conflicts in your grammar.

15.2 The E language

Add a `Eglobvar` expression of `string` to the expressions `E`. The `Eglobvar` "max" expression contains the memory address of the global variable named "max". Indeed, all global variables are stored in memory. We will see later how to initialize them.

15.3 Generation of E

In the same way that you added a parameter `typ_struct` for structures, add a parameter `typ_glob : (string, typ) Hashtbl.t` to each function that needs it (`type_expr`, `make_eexpr_of_ast`, `make_einstr_of_ast` ...).

You will need to differentiate, when you encounter a variable identifier, whether it is a local variable (`Evar v`) or a global variable (`Eglobvar v`). Use the type information (`typ_var` and `typ_glob`) to do this.

The initialization of a global variable must be a constant, of integer or character type. In the absence of an initializer, simply reserve space.

To characterize this, add the following type to `src/prog.ml`:

```
type init_data =
  | Iint of int
  | Ichar of char
  | Ispace of int
```

We will also change the definition of type `gdef` to add a constructor `Gvar` of `typ * init_data`.

For example, the two global definitions from the example at the beginning of the section will generate the following global definitions: `Gvar(Tint, Iint 20)` and `Gvar(Ttab(Tint, 20), Ispace(160))` (because $20 \times 8 = 160$).

Modify the function `make_fundef_of_ast` to handle global variable definitions. In the process, rename it to `make_globdef_of_ast` and change its return type by replacing `(string * efun)` with `(string * efun gdef)`.

15.4 E Interpreter

Add a field `glob_env`: `(string, 'a) Hashtbl.t` to the type state in `src/prog.ml`, which will collect the addresses of the different global variables.

Finally, add, still in this file, a function

```
let init_glob
  (mem: Mem.t) (glob_env: (string, int) Hashtbl.t)
  (p: 'a prog) (startglob: int) : int res = ...
```

which, given a program `p` and a starting address `startglob`, extends the global environment `glob_env` and writes to memory `mem`. To do this, iterate over the definitions of the program `p` (ignoring function definitions) and write the initialization integer or character of the global in question, or simply reserve space if the initializer is `Ispace n`.

Add in `src/elang_run.ml` a parameter `typ_glob` wherever it seems necessary to you. The evaluation of the expression `Eglobvar v` should be similar to the case `Evar v`, except that you will look in the global environment rather than in the local environment and that the variables are always allocated in memory.

In the function `eval_prog`, when you iterate over the program definitions and come across a global variable, you need to update its type in the table `typ_glob`. Also, call the function `init_state` that you defined in `src/prog.ml`. You can use, for example, the address `0x1000` as the starting address for globals. (The choice of this address doesn't really matter. It just needs to provide enough space to store all the global variables and not conflict with the stack. When you launch your compiler, the memory size is by default 10000 (i.e., `0x2710`). Allocating globals at `0x1000` should not cause any problems for any of the tests provided. The "real" allocation, for RISC-V programs, will be performed by the linker, with greater separation between the different memory areas.)

15.5 Generation of CFG

Repeat the same operations as for the E interpreter: add a field `typ_glob`, add an expression `Eglobvar of string`, extend `typ_glob` in the function `cfg_prog_of_eprog` to add the type of the globals there.

15.6 Other intermediate languages

For RTL, add an operation `Rglobvar of reg * string`. The operation `Rglobvar(rd, v)` puts the address of the global `v` into register `rd`.

For LTL, add a `LGlobvar of ltl_reg * string` operation, this time with a machine register.

For each compilation pass, it is also necessary to propagate the global definitions. It is enough to leave them unchanged (no need to compile them).

We will also add to the state of RTL and Linear programs a field `glob_env` of the same nature as that defined for E and CFG in the type `state of src/prog.ml`, and similarly for LTL programs in `src/ltl_run.ml`.

In `src/riscv.ml`, compile the operation `LGlobvar(rd, s)` using the RISC-V instruction `la rd, symbol`, which writes the address of the symbol `symbol` into the register `rd`.

Still in `src/riscv.ml`, you must declare the global variables that you use in a `.data` section, which is declared by writing `.section .data`. For each variable, if it is initialized, you must provide its content as a list of bytes (you can use the function `split_bytes`, already described in this document). If it is not initialized, you will put null bytes there.

For example:

```
.section .data
max:
    .byte 20,0,0,0,0,0,0,0,0
objs:
    .zero 160
```

16 Space Invaders!

Expected test coverage

The individual language features needed by the game should pass `ecomp-tests/invader_features/`. The complete integration program in `ecomp-tests/invader/invader.e` is the final check, so all applicable earlier suites should remain passing too.

To conclude this compilation project in a grand finale, we suggest you play a cutting-edge video game (1978) from the Japanese company Taito: *Space Invaders* (スペースインベーダ).

You will find the source code for this game in `ecomp-tests/invader/invader.e`. New language constructs will prevent you from compiling it directly. It is up to you to figure out how to handle these new constructs.

16.1 New language constructions

In particular, you will need to address:

- hexadecimal numbers like `0x123aBc12F`. (You should only touch the `src/e_regexp.ml` file for this.)
- boolean operators like `&&` and `||`.

In actual C, the expression `e1 && e2` will be evaluated as follows. We evaluate `e1`: if its value is false (`0`), then `e2` (which may contain side effects) is not evaluated, and `0` is returned. Otherwise, `0` is returned if `e2` equals `0`, and `1` otherwise.

Similarly, when evaluating `e1 || e2`, `e2` is not evaluated if `e1` is true (`!= 0`).

That being said, as a first approximation, you can evaluate the two operands and calculate the Boolean AND / OR using the RISC-V instructions `and` and `or`, noting that this only works when the expressions `e1` and `e2` evaluate to `0` or `1`. Unless there is an error on our part, the code in `invader.e` and the code in `src/riscv.ml` adhere to this convention.

- the bitwise AND operator `&`. You can use the function `land : int -> int -> int` which performs this operation on OCaml integers, and the RISC-V instruction `and`.
- the logical NOT operator `!`. You will use the RISC-V instruction `seqz rd, rs` which writes the value `1` into `rd` if `rs` is `0`, and the value `0` into `rd` otherwise.

16.2 Compilation

The first step in compiling this program is to output assembly code.

In the directory `ecomp-tests/invader/`:

```
$ ../../ecomp -f invader.e -m32 -nostart -clever-regalloc
```

This command produces the file `ecomp-tests/invader/invader.s` which contains the assembly code of this application.

The `-nostart` option indicates that the symbol `_start`, the program entry point, must not appear in the generated code. Indeed, it appears in a library that we are going to link to our program right away:

```
$ riscv64-unknown-elf-gcc -march=rv32im_zicsr -mabi=ilp32 -mmodel=medany \
  -I. -I./include -g -DENV_QEMU=1 -T ./extended.lds \
  -nostartfiles -nostdlib -nostdinc -static \
  invader.s crt.s setup.c font.c libscreen.c libfemto.a itoa.c \
  -o invader
```

You should obtain a RISC-V executable, which you can verify in the following way:

```
$ file invader
invader: ELF 32-bit LSB executable, UCB RISC-V, version 1 (SYSV), statically
linked, with debug_info, not stripped
```

16.3 Execution

You have an executable, but you don't know how to run RISC-V binaries. We will use a RISC-V processor emulator.

We have prepared a specific installation of `qemu` for you in a Docker container.

To install Docker and a VNC viewer (we'll see why in a few minutes) on Ubuntu or Debian:

```
$ sudo apt install docker.io xtightvncviewer
```

You can then retrieve the docker image prepared for you:

```
$ docker pull pwilke/qemu-compil:latest
```

Now go to the directory `ecomp-tests/invader`, where you generated the RISC-V binary `invader`.

```
$ docker run --rm -v $(pwd):/data -p 5900:5900 -it pwilke/qemu-compil:latest invader
```

Ah, but it doesn't display anything!

The image is sent to a VNC (Virtual Network Computing) socket. (We could probably display it directly on your screen, but not portably.)

It is therefore necessary to connect with a VNC client.

If you are using Linux, you installed a VNC client above; you only need to run:

```
$ vncviewer 127.0.0.1:5900
```

If you are on Windows or Mac OS, download a VNC client (<https://www.clubic.com/telecharger-fiche46296-vnc-viewer.html> looks great.)

Hurrah!

1 Debugging programs at the low-level language level

A.1 : Debugging a program at the LTL language level

You can make the LTL interpreter go into debug mode using `ecomp-run's -ltl-debug` command-line switch. In that case, the runner waits for debugging commands coming from an external program on a websocket⁵ listening on IP address `127.0.0.1` on port `8080`.

We provide an external program that acts as a debugger. It is written in JavaScript, and its code is located in the directory `ldb/`.

Thus, to debug at the LTL language level, first save the LTL artifact produced by the compiler, then load it in the runner. For example:

```
$ ./ecomp -f ../ecomp-tests/basic/gcd.e -ltl-save /tmp/gcd.ecomp-ltl  
$ ./ecomp-run -ltl-load /tmp/gcd.ecomp-ltl -ltl-debug
```

Then open `ldb/ldb.html`.

You will then see a window that looks like the one shown in [Figure 23](#).

⁵A websocket is a TCP socket that can be opened from a web browser to connect to a remote site with security restrictions. The TCP stream transports a dedicated protocol that can be decoded as JSON objects from a JavaScript program running in the browser.

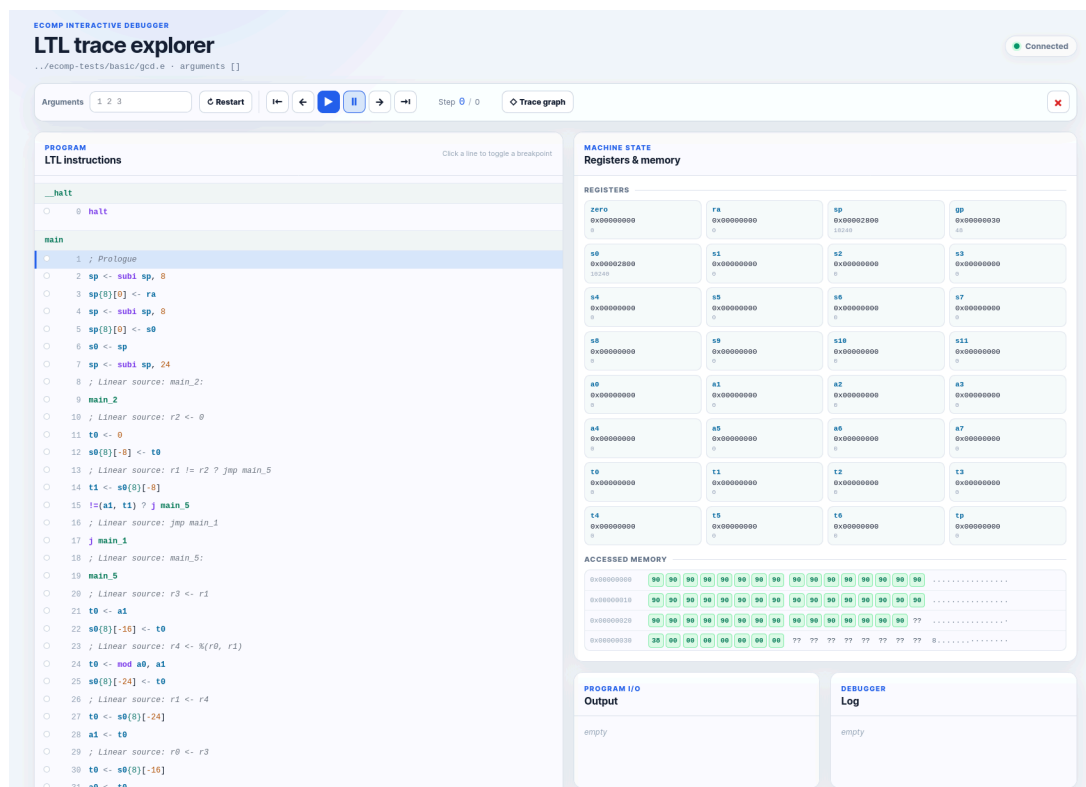


Figure 23: The LDB debugger interface

This window allows you to control the LTL interpreter provided by `ecompile-run`.

A.2 : Debugging a program at the RISC-V language level

To debug a generated RISC-V executable, start QEMU in GDB server mode in one terminal. For example, for a 64-bit program:

```
$ qemu-riscv64 -g 1234 ../ecompile-tests/basic/gcd.exe 54 24
```

QEMU waits for a debugger to connect on TCP port 1234. In another terminal, start the matching cross-debugger and connect to it:

```
$ riscv64-unknown-elf-gdb ../ecompile-tests/basic/gcd.exe
(gdb) target remote :1234
(gdb) break main
(gdb) continue
```

You can then use ordinary GDB commands such as `stepi`, `nexti`, `info registers`, and `x/8gx $sp`. For a 32-bit executable, use `qemu-riscv32` and the corresponding 32-bit RISC-V cross-debugger.